DOCUMENTAÇÃO TÉCNICA

Componente Curricular de ASG e ODS

Portal institucional e manifestação de interesse

Conselho Regional de Administração de São Paulo
CRA-SP

Edição 1.0 · 13 de outubro de 2026

Repositório: desenvolvimentoprofissional-oss/asg-ods

Código de referência: main · 9a126c6

Revisão completa: 9a126c65ba902e19b31bc695be0ccc172df7167f

Sumário

1. Resultados externos e identificação do documento....4
1.1 Quadro de resultados dos scanners....4
1.2 Indicadores complementares apresentados....4
1.3 Identificação e metodologia documental....5
2. Visão do produto e arquitetura....6
2.1 Finalidade e público....6
2.2 Estrutura do componente curricular....6
2.3 Mapa de páginas e percurso de navegação....7
2.4 Composição da interface....8
2.5 Catálogos, textos e identidade visual....9
2.6 Interações de conteúdo e recursos globais....9
2.7 Fronteira entre navegador e servidor....9
2.8 Manifestação de interesse como ligação entre as camadas....10
3. Interface web, navegação e interação....11
3.1 Montagem e composição da aplicação....11
3.2 Rotas e páginas....11
3.3 Layouts e navegação institucional....12
3.4 Formulário e seus componentes....12
3.5 Envio, mensagens e confirmação....13
3.6 Preferências de acessibilidade e sincronização visual....14
3.7 Diálogo de governança e privacidade....14
3.8 Navegação por teclado para apresentação....14
3.9 Inventário funcional de componentes....15
4. Backend, API e processamento das requisições....16
4.1 Organização do servidor....16
4.2 Ordem dos middlewares....16
4.3 Rotas e entrega de arquivos....17
4.4 Contrato de entrada do cadastro....17
4.5 Respostas HTTP e integração com o cliente....17
4.6 Caminho completo de uma manifestação....18
5. Configuração Firebase, organização dos dados e criptografia....19
5.1 Seleção do projeto e da base Firestore....19
5.2 Coleções e vínculos entre registros....19
5.3 Documentos agregadores e cálculo de capacidade....20
5.4 Transação, unicidade e reconstrução do índice....21
5.5 HMAC, cifragem e formato do cofre....21
5.6 Mensagens operacionais e sanitização....21
6. Protocolos HTTP e recursos de distribuição....23
6.1 Cabeçalhos definidos pelo backend....23
6.2 Organização da Content Security Policy....23
6.3 Integridade dos assets e distribuição estática....24
6.4 Manifestos e endereços públicos....24
7. Operação, configuração e testes....25

7.1 Runtime e instalação....25
7.2 Comandos disponíveis....25
7.3 Desenvolvimento, build e produção....26
7.4 Firebase Hosting e Cloud Run....26
7.5 Variáveis e credenciais....27
7.6 Organização dos testes....28
7.7 Métricas e geração de relatórios....28
7.8 Agregado de código e arquivos públicos....29
7.9 Catálogo operacional de arquivos....29
8. Glossário e referências de implementação....31
8.1 Termos utilizados....31
8.2 Fontes de implementação....31
8.3 Referências das ferramentas externas....32
Anexo A. Páginas da aplicação — desktop....33
A.1 Página inicial....33
A.2 Chancela institucional....34
A.3 Metodologia dinâmica....35
A.4 Objetivos de aprendizagem....36
A.5 Manifestação de interesse....37
Anexo B. Páginas da aplicação — mobile....38
B.1 Página inicial e Chancela institucional....38
B.2 Metodologia dinâmica e Objetivos de aprendizagem....39
B.3 Manifestação de interesse....40

1. Resultados externos e identificação do documento

1.1 Quadro de resultados dos scanners

O quadro reúne os resultados apresentados nas capturas fornecidas pelo responsável pelo projeto. O endereço avaliado é **https://asg-ods.ai.studio/**. As classificações e os indicadores foram transcritos com suas unidades e identificação de ferramenta; as datas distinguem criação de relatório e captura de tela.

Ferramenta e modalidade	Resultado apresentado	Data de referência
MDN HTTP Observatory	**A+**; score **150/100**; **11/12** testes aprovados.	Captura: 13/10/2026.
PageSpeed Insights — mobile	Desempenho **99**; acessibilidade **100**; práticas recomendadas **96**; SEO **100**; navegação agêntica **4/4**.	Relatório: 13/10/2026, 15:41:05.
PageSpeed Insights — desktop	Desempenho **100**; acessibilidade **100**; práticas recomendadas **96**; SEO **100**; navegação agêntica **4/4**.	Relatório: 13/10/2026, 15:41:05.
SecurityHeaders	**A+**. O resumo apresenta HSTS, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, CSP e X-Frame-Options.	Relatório: 13/10/2026, 15:41:10 (UTC−3).
Catchpoint / WebPageTest	FCP **0,716 s**; LCP **0,716 s**; CLS **0**; TTFB **0,266 s**; TBT **0 s**.	Captura: 13/10/2026.
WAVE / WebAIM	**AIM Score 10/10**; **0** erros; **0** erros de contraste; **0** alertas.	Captura: 13/10/2026.
Webbkoll	HTTPS por padrão; **Good policy**; **Referrers not leaked**; **1** cookie próprio, **0** de terceiros; **2** requisições a **2** hosts externos.	Relatório: 13/10/2026, 15:17:00 (UTC−3).

Os horários originalmente informados em UTC no SecurityHeaders e no Webbkoll foram convertidos para America/Sao_Paulo. No PageSpeed, foi mantido o horário exibido na captura. Observatory, WebPageTest e WAVE recebem a data da captura quando o recorte não apresenta uma data absoluta do relatório. A pontuação 150/100 do Observatory foi preservada como apresentada.

1.2 Indicadores complementares apresentados

A captura do WebPageTest apresenta o conjunto a seguir. FCP identifica a primeira renderização de conteúdo; LCP, a renderização do maior conteúdo; CLS, a variação acumulada de layout; TTFB, o tempo até o primeiro byte; TBT, o tempo total de bloqueio. Os nomes DC Time e DC Bytes são mantidos conforme o painel da ferramenta.

Indicador	Valor apresentado
First Contentful Paint	0,716 s
Largest Contentful Paint	0,716 s
Cumulative Layout Shift	0
Time To First Byte	0,266 s
Start Render	0,7 s
Speed Index	0,79 s
Total Blocking Time	0 s

Indicador	Valor apresentado
Page Weight	345 KB
DC Time	0,793 s
DC Bytes	330 KB
Total Time	1,56 s
Total Requests	30

A captura WAVE também apresenta um recurso identificado, três elementos de estrutura e zero elementos ARIA no resumo mostrado. O Webbkoll registra a passagem do endereço HTTP para o endereço final HTTPS. No PageSpeed, a área de experiência dos usuários mostra “Nenhum dado”, e os valores numéricos exibidos ficam na área de diagnóstico de desempenho.

1.3 Identificação e metodologia documental

A documentação descreve a implementação do repositório **desenvolvimentoprofissional-oss/asg-ods**, branch **main**, na revisão **9a126c65ba902e19b31bc695be0ccc172df7167f**. A edição é **1.0**, de **13 de outubro de 2026**, em português do Brasil. O domínio público indicado nos metadados do projeto é **asg-ods.ai.studio**.

A organização segue uma descrição por camadas: contexto e finalidade do produto; composição da interface; comunicação entre navegador e servidor; contrato HTTP; organização dos dados; configuração e execução; ferramentas e testes. Os diagramas representam o contexto, a composição e os fluxos implementados. Essa sequência aproxima a apresentação do produto de seus elementos de código e de sua operação.

As fontes de implementação são os arquivos mantidos no checkout. Contratos e dicionários de dados são extraídos de schemas, tipos e construtores; comandos são extraídos de package.json; rotas e comportamentos são extraídos dos componentes e do servidor. Os resultados externos têm como fonte as capturas recebidas, separadamente da descrição do código. As referências ao final permitem localizar as origens utilizadas.

A leitura pode seguir o percurso completo ou começar pela área de interesse. Os capítulos da interface organizam o uso do portal; os capítulos de API e dados apresentam a manifestação de interesse; o capítulo operacional descreve instalação, execução, publicação e utilitários. O sumário e os marcadores do PDF oferecem navegação entre essas áreas. Os anexos apresentam capturas do build dessa revisão em desktop e mobile, com títulos e legendas de identificação.

2. Visão do produto e arquitetura

2.1 Finalidade e público

O portal apresenta o Componente Curricular de ASG e ODS, associado ao Conselho Regional de Administração de São Paulo, CRA-SP, e às instituições participantes. ASG corresponde às dimensões Ambiental, Social e Governança; ODS identifica os Objetivos de Desenvolvimento Sustentável. A comunicação da aplicação situa esses temas na formação em Administração e Gestão e apresenta uma proposta de incorporação às matrizes curriculares das Instituições de Educação Superior, IES.

O público compreende professores, coordenadores, diretores e demais representantes acadêmicos envolvidos na implementação do componente. A página de registro oferece opções de cargo e curso para contextualizar a manifestação de interesse. O visitante percorre a apresentação institucional, conhece os formatos de aprendizagem, consulta os objetivos por módulo e informa a previsão de implementação em sua instituição.

A proposta apresentada é gratuita e modular. Os textos permitem compreender a adoção integral, a utilização de módulos em disciplinas existentes e o emprego do material como apoio docente. O fluxo de manifestação de interesse reúne dados para contato posterior: a mensagem de conclusão informa que a equipe do CRA-SP entrará em contato para disponibilizar orientações quando os conteúdos estiverem disponíveis.

2.2 Estrutura do componente curricular

O catálogo curricular contém seis módulos de quatro horas, totalizando 24 horas. Cada módulo associa título, instituição participante, descrição resumida, ícone e objetivos detalhados. A identificação numérica mantém a sequência entre a apresentação resumida da página inicial e a tabela da trilha.

Módulo	Tema	Instituição indicada	Carga
1	Histórico da RSC e do ASG	CRA-SP	4h
2	Dimensão Ambiental	ANGRAD	4h
3	Dimensão Social	PRME	4h
4	Dimensão Governança	IBGC	4h
5	Integridade Socioambiental	ETHOS	4h
6	Mensuração e Investimento	ANGRAD	4h

Os objetivos estão organizados por nível de Bloom, verbo de ação, descrição e carga horária. No catálogo dessa versão, cada módulo reúne quatro objetivos de uma hora. Os assuntos percorrem evolução conceitual, sustentabilidade, relações sociais, estruturas de governança, integridade e comunicação do valor ASG.

Essa estrutura é declarada em `modules.ts` e `modulesDetails.ts`. O primeiro arquivo reúne a identidade e o resumo dos módulos; o segundo contém as linhas exibidas nas tabelas. A página inicial utiliza o resumo, enquanto a trilha apresenta também o detalhamento. A mesma configuração alimenta as duas formas de apresentação.

2.3 Mapa de páginas e percurso de navegação

A aplicação possui cinco páginas principais. React Router associa os caminhos aos componentes correspondentes, e os links internos conduzem o visitante entre apresentação, detalhamento e formulário.

Caminho	Página	Conteúdo principal
`/`	Início	Apresentação, destaques, módulos, interesse, FAQ e parceiros
`/chancela`	Chancela Institucional	Manifesto, mensagem institucional e instituições signatárias
`/metodologia`	Metodologia Dinâmica	Pilares, formatos e opções de implementação
`/trilha`	Objetivos de Aprendizagem	Sequência dos módulos e tabelas de objetivos
`/registro-interesse`	Manifestação de Interesse	Dados da instituição, contato e previsão de implementação

A página inicial começa com imagem e chamada principal. Em seguida, apresenta cartões de acesso à chancela e à metodologia, objetivos resumidos, outra chamada para registrar interesse, perguntas frequentes e identificação das instituições participantes. As chamadas de interesse apontam para o mesmo formulário.

A chancela reúne um banner, o relato do manifesto e uma mensagem atribuída ao presidente do CRA-SP. O carrossel final apresenta descrições e logos de CRA-SP, ANGRAD, Instituto Ethos, IBGC e PRME. A metodologia organiza sua exposição em quatro pilares: Taxonomia de Bloom, Microlearning, Conexão com o Mercado e Sala de Aula Invertida. Os formatos apresentados são videoaulas, podcasts e material de apoio; as opções de implementação incluem adoção integral, inserção modular e apoio docente.

A trilha utiliza uma linha de sequência visual e cartões numerados. Cada cartão contém resumo, instituição, duração e tabela. Os identificadores `modulo-1` a `modulo-6` permitem links diretos a essas seções. Os caminhos `/login`, `/cadastro` e `/area-docente/*` encaminham para a manifestação de interesse. Os demais caminhos recebem o encaminhamento previsto para a página inicial.

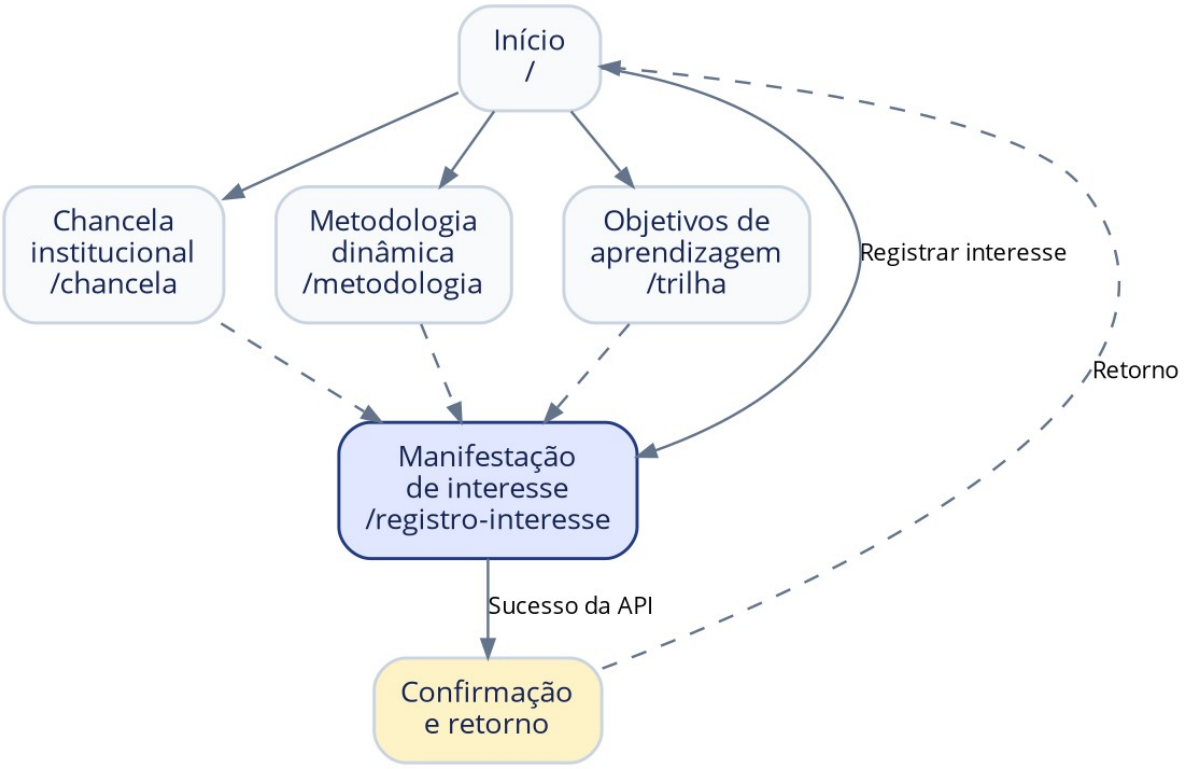


Figura 1 — Páginas e percurso de manifestação de interesse.

2.4 Composição da interface

`src/main.tsx` inicia a aplicação React no elemento `root` de `index.html`. `App.tsx` reúne o roteador, a sincronização das preferências de apresentação, a composição de páginas e os componentes globais de apresentação e Libras. Parte dos componentes é carregada por importação dinâmica, com `Suspense` organizando o conteúdo durante esse carregamento.

As quatro páginas informativas compartilham `PublicLayout`: barra superior, cabeçalho, conteúdo principal e rodapé. O cabeçalho oferece as rotas institucionais; o rodapé reúne links temáticos, contato e localização. O layout também atribui o título do documento conforme o caminho e interpreta os fragmentos usados para chegar a uma seção. A manifestação de interesse utiliza uma composição própria, com área institucional e formulário.

Os arquivos de `src/pages` compõem blocos visuais menores. As pastas `home`, `chancela`, `metodologia` e `trilha`, dentro de `src/components`, agrupam componentes pelo contexto em que aparecem. Essa organização permite localizar um banner, uma tabela ou uma seção a partir da página que os apresenta.

React define a composição e os estados da interface; React Router organiza a navegação; Tailwind CSS fornece classes de apresentação; Lucide React fornece ícones. Os componentes básicos `Button`, `Card`, `Badge` e `SectionHeader` recebem conteúdo e propriedades para adaptar aparência e ação. Variantes, tamanho, classes adicionais, filhos e callbacks são utilizados na composição das páginas.

2.5 Catálogos, textos e identidade visual

Os conteúdos estáticos ficam concentrados em `src/constants`. `navigation.ts` define os itens principais do cabeçalho; `footer.ts` reúne os grupos do rodapé; `content.ts` mantém perguntas e respostas; `courses.ts` fornece as opções de cargo e curso do formulário. Os textos da metodologia estão em `metodologia.ts`, e o cabeçalho da trilha está em `trilha.ts`.

`partners.ts` descreve os logos usados nas faixas e na apresentação dos participantes. `chancela.ts` reúne as instituições signatárias, suas descrições, imagens e configurações visuais. Os logos dessas instituições são referenciados em `/imgs/partners`, dentro de `public`. Imagens de banners e cartões completam a identidade visual das páginas.

Os componentes recebem entradas desses catálogos por propriedades e percorrem as listas para montar cartões, itens e tabelas. Nome, imagem, texto e instituição conservam sua relação na configuração de origem. O CSS global, em `src/index.css`, organiza o contêiner de página, animações, comportamento visual dos controles e estilos associados às preferências de apresentação.

2.6 Interações de conteúdo e recursos globais

O FAQ utiliza `FaqAccordion` e `FaqItem`. Inicialmente, exibe as três primeiras perguntas; o comando “Ver mais perguntas” apresenta a lista completa, e “Ver menos” retorna à composição inicial. A seleção de uma pergunta abre sua resposta, mantendo um índice de item aberto. Botões e regiões de resposta possuem identificadores que relacionam comando, estado e conteúdo.

O carrossel das signatárias oferece controles anterior e próximo, marcadores de posição e rolagem horizontal. A configuração é repetida em três conjuntos para formar o percurso contínuo. O hook `useSignatoriesCarousel` coordena dimensões, posição ativa, balanceamento da rolagem e avanço automático a cada cinco segundos. A animação de deslocamento utiliza `requestAnimationFrame`.

Na página inicial, uma faixa movimenta os logos dos participantes; a seção final utiliza a lista estática de parceiros. O componente de apresentação organiza uma sequência entre início, chancela, metodologia e trilha. As teclas `f` e `p` alternam tela cheia, enquanto setas e PageUp/PageDown percorrem essa sequência nas condições definidas pelo componente.

`VLibrasWidget` cria a região “Acessibilidade em Libras”, carrega o plugin de `vlibras.gov.br` e instancia o widget com a URL de sua aplicação. A constante de identificação da integração é `7.12.2`. O componente mantém controles de tentativa e temporizadores e possui uma mensagem de estado para indisponibilidade. As preferências da barra superior são mantidas pelo estado de apresentação e sincronizadas com classes e atributos do documento.

2.7 Fronteira entre navegador e servidor

A arquitetura reúne cliente web e servidor Node.js no mesmo repositório. O cliente apresenta os catálogos, organiza a navegação, mantém o estado do formulário e envia a manifestação de interesse. O servidor Express recebe a operação HTTP, valida sua estrutura e conduz a persistência por Firebase Admin e Firestore.

`src/types/leads.ts` contém o contrato TypeScript e o schema Zod da manifestação. `src/api/leadsService.ts` implementa a chamada `POST /api/registrar-interesse`, serializa o corpo JSON e interpreta a resposta. O serviço distingue a resposta de duplicidade das demais respostas de erro e entrega o resultado ao fluxo da interface.

`server.ts` organiza os middlewares e a rota de registro. As funções de `src/server` distribuem as responsabilidades de persistência, construção dos registros e cabeçalhos HTTP. A preparação dos registros operacionais e dos dados de contato ocorre nesse lado da arquitetura. O navegador comunica-se com a API da aplicação, e a integração administrativa com Firestore é realizada pelo servidor.

Em desenvolvimento, o servidor integra os middlewares do Vite. Em produção, entrega os arquivos compilados de `dist` e o documento de entrada da aplicação para as rotas do cliente. Os assets em `public` participam da distribuição estática. Assim, o mesmo processo conecta a entrega da interface ao endpoint usado pelo formulário.

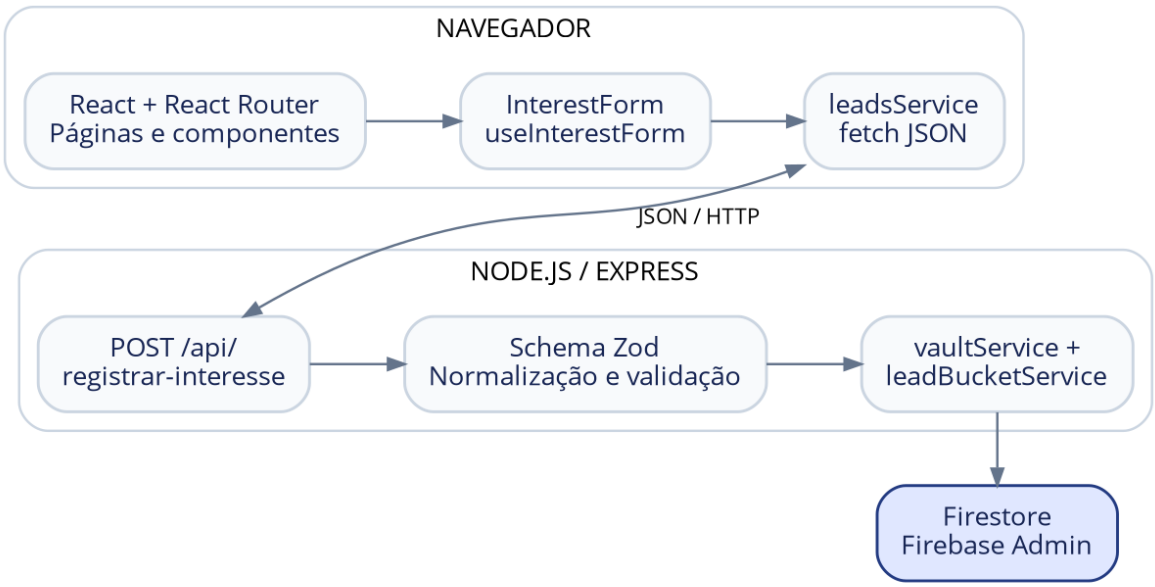


Figura 2 — Comunicação entre navegador, API Express e Firestore.

2.8 Manifestação de interesse como ligação entre as camadas

O formulário agrupa campos de instituição, campus, pessoa de contato, cargo, e-mail, telefone, curso, previsão e observações. `useInterestForm` mantém esses valores, a situação de envio e a mensagem de erro. Na submissão, reúne o payload e chama o serviço de API. Durante o processamento, o comando principal indica “Registrando…”.

A página recebe a conclusão por callback e passa a apresentar a mensagem de interesse registrado, com o primeiro nome do contato e um link de retorno à página principal. Esse percurso conecta o conhecimento da proposta curricular ao registro dos dados necessários para a continuidade do contato institucional.

3. Interface web, navegação e interação

Este capítulo descreve a interface React do Componente Curricular de ASG e ODS, tomando como referência o commit `9a126c65ba902e19b31bc695be0ccc172df7167f`. A aplicação organiza a apresentação institucional em páginas públicas e concentra a manifestação de interesse em uma tela própria. Componentes visuais, estado de formulário, comunicação com a API e preferências de acessibilidade têm responsabilidades específicas dentro dessa composição.

3.1 Montagem e composição da aplicação

O documento `index.html` define idioma `pt-BR`, metadados, título inicial e o elemento `root`. Dentro desse elemento existem cabeçalhos estáticos com o nome do componente curricular e do CRA-SP. A entrada JavaScript está em `src/main.tsx`: ela importa a folha global `src/index.css`, cria a raiz React com `createRoot` e renderiza `App` dentro de `StrictMode`. A montagem React passa a fornecer o conteúdo da interface nesse ponto do documento.

`src/App.tsx` compõe quatro responsabilidades: roteamento, posição de rolagem, sincronização visual das preferências e recursos complementares. `BrowserRouter` oferece o contexto de navegação usado pelos links e páginas. `ScrollToTopHost` observa o caminho atual e executa a rolagem para o início quando ele muda. `AccessibilitySyncHost` mantém as preferências de fonte e contraste aplicadas ao elemento HTML durante a navegação.

`AppRoutes` fornece as telas. `PresentationMode` e `VLibrasWidget` são carregados com `lazy` e reunidos em um `Suspense` com conteúdo de espera nulo. O primeiro instala ações de teclado; o segundo compõe o recurso de tradução em Libras. A organização permite que esses recursos sejam montados na composição geral da aplicação, enquanto as páginas continuam descritas no módulo de rotas.

3.2 Rotas e páginas

O mapa de navegação está em `src/routes/index.tsx`. A home é importada diretamente; Metodologia, Chancela, Trilha e Registro de Interesse usam carregamento por `lazy`. Durante o carregamento de uma página, `PageFallback` apresenta um indicador circular dentro de uma região com `aria-busy="true"`.

Endereço	Componente ou destino	Papel no fluxo
`/`	`Home`	Apresentação geral e entradas para os conteúdos.
`/chancela`	`Chancela`	Apresentação da chancela e dos signatários.
`/metodologia`	`Metodologia`	Organização da metodologia e de sua implementação.
`/trilha`	`Trilha`	Exposição dos módulos e objetivos de aprendizagem.
`/registro-interesse`	`InterestRegistrationPage`	Preenchimento e confirmação da manifestação.
`/login`, `/cadastro`	Redirecionamento para registro	Entradas alternativas para o mesmo formulário.
`/area-docente/*`	Redirecionamento para registro	Encaminhamento dos caminhos dessa família.
Demais caminhos	Redirecionamento para `/`	Convergência para a página inicial.

Os redirecionamentos usam `Navigate` com `replace`, substituindo a entrada corrente do histórico. O botão “Entrar” da barra superior aponta para `/login` e segue esse encaminhamento até o registro. Nesse mapa, os endereços login, cadastro e área docente cumprem a função de acesso à manifestação de interesse.

As páginas públicas pertencem a `PublicLayout`. `Home` reúne hero, banners, módulos, faixa de adesão, perguntas frequentes e parceiros; parte dessas seções utiliza carregamento tardio. `Chancela` combina banner, manifesto, filtros gráficos e carrossel de signatários. `Metodologia` reúne banner, pilares, formatos e implementação. `Trilha` combina banner e linha de módulos. Cada página coordena componentes próprios de sua seção e utiliza os catálogos correspondentes em `src/constants`.

3.3 Layouts e navegação institucional

`PublicLayout` organiza a sequência `TopBar`, `Header`, conteúdo principal e `Footer`. O componente `Outlet`, dentro de `main`, recebe a página escolhida pelo router. O layout define títulos de navegador por caminho: início, chancela institucional, metodologia e diretrizes, e trilha de aprendizagem. Quando a localização contém um fragmento, procura o elemento pelo identificador e agenda uma rolagem suave para ele. Nas navegações sem fragmento, posiciona a página no início.

`TopBar` reúne ferramentas visuais, redes sociais e o acesso “Entrar”. Seus componentes distribuem os botões de contraste, tamanho de fonte e privacidade, mantendo as funções de cada controle conectadas ao estado correspondente. `TopBarSocial` descreve os links sociais e seus nomes acessíveis. O rodapé reúne endereço, contato, ouvidoria, acesso à informação, redes sociais e colunas de links institucionais.

O cabeçalho apresenta o símbolo CRA-SP como link para a home. `HeaderDesktopNav` e `HeaderMobileMenu` compartilham `headerNavigation`, em `src/constants/navigation.ts`, com três destinos: Chancela Institucional, Metodologia Dinâmica e Objetivos de Aprendizagem. A comparação entre o caminho atual e o destino do item identifica a página selecionada; o link correspondente recebe `aria-current="page"`.

Em telas móveis, `Header` mantém `isMobileMenuOpen` como estado local. O botão alterna a abertura, troca seu ícone e atualiza `aria-expanded`, nome acessível e referência `aria-controls`. O menu é renderizado quando esse estado está ativo. Selecionar um item executa a navegação e a função de fechamento fornecida pelo cabeçalho.

`InterestRegistrationPage` usa uma composição própria: a coluna `LoginBrandingColumn` apresenta a identidade visual em telas grandes, e a área principal contém o retorno ao início e o formulário ou sua confirmação. Essa página estabelece o título “Manifestação de Interesse | CRA-SP” ao montar e posiciona a rolagem no início.

3.4 Formulário e seus componentes

`InterestForm` organiza o título, a região de mensagem, os grupos de campos e o botão de envio. Os componentes de campo recebem o valor atual e funções de atualização. Assim, os controles são conectados ao mesmo conjunto de dados administrado pelo hook `useInterestForm`.

Grupo	Dados apresentados	Componentes
Instituição	Instituição de Educação e campus/unidade	`InstitutionFields`, `IesInput`, `CampusInput`.

Grupo	Dados apresentados	Componentes
Contato	Nome completo, cargo/função, email institucional e telefone	`ContactFields` e seus controles individuais.
Planejamento	Curso principal, semestre pretendido e observações	`CourseFields`, `CourseSelect`, `SemesterSelect`, `ObservationsRow`.

Instituição, campus, nome, email, telefone e previsão são obrigatórios. Cargo, curso e observações completam o contexto informado pelo interessado. O semestre é selecionado entre o primeiro e o segundo semestre de 2027. Os catálogos de cargo e curso estão em `src/constants/courses.ts`. Os controles associam seus rótulos aos identificadores dos campos, e nome, email, telefone e instituição usam os tokens de preenchimento `name`, `email`, `tel` e `organization`.

O hook inicia os nove campos com strings vazias. Também mantém `isLoading` e `error`. A função `update` recebe o nome de um campo e seu novo valor, produzindo um objeto de estado que preserva os demais dados. Esse objeto alimenta novamente os componentes visuais após cada atualização.

O telefone utiliza `formatPhone`, em `interestFormUtils.ts`. A função remove caracteres não numéricos e, quando há mais de onze dígitos iniciados por 55, retira esse prefixo. Em seguida, trabalha com até onze dígitos e compõe a apresentação com DDD, espaço e hífen. A edição acompanha os comprimentos intermediários; números completos usam a organização de quatro ou cinco dígitos antes do hífen. O controle admite entrada de até vinte caracteres e chama a normalização em cada alteração.

3.5 Envio, mensagens e confirmação

O envio é coordenado por `handleSubmit`, em `useInterestForm`. A função intercepta o evento do formulário e verifica se os campos obrigatórios têm conteúdo após remoção de espaços externos. Quando algum deles está vazio, define a mensagem “Por favor, preencha todos os campos obrigatórios.”.

Para iniciar a comunicação, o hook ativa `isLoading` e limpa a mensagem anterior. `buildPayload` prepara os dados: aplica `trim` a nome, email, telefone, instituição, campus e observações; mantém os valores selecionados de cargo e curso; e transfere `previsao` para a propriedade `previsaoSemestre`. O payload segue o tipo compartilhado `LeadRegistrationInput`.

`registrarManifestacaoInteresse`, em `src/api/leadsService.ts`, envia um `POST` para `/api/registrar-interesse`, com `Content-Type: application/json` e corpo serializado. A função lê o corpo da resposta como JSON. O status 409 gera `DuplicateLeadError`. Nos demais casos, o retorno de sucesso depende de resposta HTTP bem-sucedida, corpo presente e propriedade `success` igual a `true`. A função então disponibiliza `success` e a mensagem recebida ou o texto padrão de confirmação.

As exceções chegam ao tratamento do hook. `handleInterestError` reconhece duplicidade e apresenta a mensagem de manifestação já cadastrada para o email institucional. Mensagens características de falha de conexão recebem orientação em português para verificar a internet e tentar novamente. Outros erros utilizam a mensagem disponível ou o texto padrão de erro de envio. `FormErrorNotice` renderiza a mensagem como alerta com anúncio assertivo.

Durante a requisição, `FormSubmitButton` fica desabilitado e apresenta “Registrando…”. Ao finalizar o processamento, o hook desativa o carregamento. Quando o serviço retorna sucesso, chama

`onSuccess` com nome e instituição. `InterestRegistrationPage` guarda esses dados em `submittedData` e passa a renderizar `InterestSuccessView`.

A confirmação mostra “Interesse Registrado!”, extrai o primeiro nome para a saudação e apresenta o agradecimento. O texto informa que a equipe CRA-SP entrará em contato para orientar a adesão quando os conteúdos estiverem disponíveis. `SuccessHomeButton` oferece o retorno à página principal. Esse encadeamento liga a mudança de tela à conclusão bem-sucedida da chamada à API.

3.6 Preferências de acessibilidade e sincronização visual

`src/store/useAccessibilityStore.ts` implementa um store externo conectado ao React por `useSyncExternalStore`. O estado contém `fontSizeScale` e `isHighContrast`. Os valores iniciais são escala 1,0 e contraste desligado. Na inicialização, a leitura de `localStorage` busca o JSON da chave `crasp_a11y_state`; a interpretação da escala aceita valores numéricos finitos entre 1,0 e 1,2. A leitura ou interpretação que gera exceção utiliza o estado padrão.

As ações públicas são `toggleHighContrast`, `increaseFontSize`, `decreaseFontSize` e `resetFontSize`. A fonte aumenta ou diminui em passos de 0,1, respeitando os limites de 1,0 e 1,2; a ação de retorno define 1,0. Cada mudança atualiza o estado corrente, comunica os assinantes e persiste o novo objeto como JSON. A assinatura retorna uma função que retira o respectivo listener do conjunto.

`useAccessibilitySync` traduz esse estado para atributos do documento: escreve `data-font-scale` no elemento HTML e alterna a classe `alto-contraste`. `src/index.css` usa a escala para calcular o tamanho base a partir de 16 pixels. Os seletores de contraste compõem fundos escuros, texto claro, destaque de links e controles, além da apresentação de foco.

Na barra superior, `TopBarFontSizeButtons` oferece A+, A− e A para aumentar, diminuir e retornar ao tamanho normal. `ContrastButton` conecta a alternância ao store e comunica seu estado com `aria-pressed`. A atualização visual e a persistência usam o mesmo estado compartilhado durante as mudanças de página.

3.7 Diálogo de governança e privacidade

`TopBar` administra `isPrivacyOpen` e mantém uma referência ao botão que abre o diálogo. `PrivacyDialog` utiliza o elemento nativo `dialog`, associado ao título “Diretrizes de Governança e Privacidade” e à descrição por identificadores ARIA.

O hook interno `useNativeDialog` sincroniza a propriedade `isOpen` com `showModal()` e `close()`. Os botões de fechar e “Entendido” compartilham o encerramento: fecham o elemento, atualizam o estado do componente proprietário e devolvem o foco ao botão de abertura. O evento de cancelamento, utilizado pelo Escape, passa pelo mesmo encerramento.

A descrição apresenta a persistência local das preferências de fonte e contraste, sua finalidade de usabilidade e a referência aos canais institucionais presentes no rodapé para informações ou requisições de titulares.

3.8 Navegação por teclado para apresentação

`PresentationMode` instala um listener de `keydown` em `window`. A sequência de páginas é início, chancela, metodologia e trilha. ArrowRight ou PageDown avança nessa sequência; ArrowLeft ou

PageUp retorna. Depois da última página, o avanço leva ao início; antes da primeira, o retorno leva à trilha.

As teclas F e P alternam tela cheia usando `requestFullscreen()` no elemento HTML ou `exitFullscreen()`. A comparação aceita letras maiúsculas e minúsculas, e essa ação considera a ausência de Ctrl e Meta. Os comandos reconhecidos utilizam `preventDefault` antes da ação correspondente.

O handler preserva a edição de controles: retorna quando o alvo é input, textarea, select, botão, conteúdo editável ou um elemento pertencente a formulário. Também retorna na rota `/registro-interesse`. O listener é atualizado com o contexto de navegação e removido no encerramento do efeito React. O componente fornece esse comportamento de teclado por sua montagem em `App`.

3.9 Inventário funcional de componentes

Grupo e caminho	Responsabilidade
`src/pages/*`	Coordenar as cinco páginas e compor suas seções.
`src/components/layout/*`	Barra superior, cabeçalho, menu móvel, marca, diálogo, rodapé e contatos.
`src/components/interest/*`	Grupos de campos, estado, normalização, mensagens e envio da manifestação.
`src/components/auth/LoginBrandingColumn.tsx`	Coluna visual da tela de registro.
`src/components/home/*`	Hero, banners, módulos, adesão, logos e perguntas frequentes da home.
`src/components/chancela/*`, `metodologia/*`, `trilha/*`	Seções visuais dos três conteúdos institucionais.
`src/components/presentation/*`	Atalhos, carrossel, cartões e indicadores de signatários.
`src/components/partners/*`, `PartnerNames.tsx`	Apresentação dos parceiros em listas e faixas.
`src/components/accessibility/VLibrasWidget.tsx`	Composição do recurso VLibras.
`src/components/ui/*`	Botões, cartões, badges e cabeçalhos de seção compartilhados.

4. Backend, API e processamento das requisições

4.1 Organização do servidor

O backend do ASG-ODS utiliza Express e Firebase Admin. O arquivo `server.ts` reúne a inicialização do servidor HTTP, a seleção da instância Firestore, os middlewares e as rotas. O processamento dos dados é distribuído entre três módulos: `leadBucketService.ts` organiza as partes e o índice de unicidade; `vaultService.ts` constrói os registros operacionais e cifra os campos do cofre; `securityHeaders.ts` define os cabeçalhos das respostas.

A aplicação Express é criada e exportada como `app`. A instância Firestore é inicializada na carga do módulo e mantida em `db`. A função `setFirestoreInstance` permite substituir essa referência dentro do processo. As funções de validação, persistência e construção de registros também possuem pontos de entrada próprios, usados pelo código e pelos testes do projeto.

O servidor utiliza a porta definida em `PORT`, com `3000` como valor de referência, e escuta em `0.0.0.0`. Quando `NODE_ENV` é `production`, o Express serve os arquivos de `dist` e encaminha as rotas da aplicação para `dist/index.html`. Em outros modos de execução, o Vite é iniciado como middleware de uma aplicação SPA. Sua configuração aceita os hosts `.ai.studio`, `.run.app`, `localhost` e `127.0.0.1`. Com `NODE_ENV=test`, a aplicação é disponibilizada sem iniciar automaticamente o listener HTTP.

4.2 Ordem dos middlewares

A configuração desabilita o cabeçalho `X-Powered-By` e estabelece `trust proxy` com valor `1`. Em seguida, registra o middleware de cabeçalhos e o parser `express.json`, cujo limite é `15kb`, equivalente a 15.360 bytes. O tratamento específico de erros JSON aparece imediatamente depois do parser. As rotas são registradas após essas etapas.

Essa ordem organiza o percurso de uma requisição: aplicação dos cabeçalhos, leitura do JSON, tratamento de erros de leitura, seleção da rota e processamento do endpoint. Na rota de cadastro, o limitador é executado antes da validação do objeto recebido. JSON malformado ou excessivo recebe sua resposta diretamente no tratamento do parser. Um corpo JSON lido corretamente segue para o limitador e para o schema.

Os cabeçalhos incluem HSTS, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer` e políticas de isolamento entre origens. A `Permissions-Policy` restringe câmera, microfone, geolocalização, pagamento, USB e browsing topics. A CSP selecionada depende de `NODE_ENV`: a política produtiva contém fontes e hashes declarados; a de desenvolvimento inclui as permissões usadas pelo Vite. O middleware também intercepta a definição de `Set-Cookie` e acrescenta `Secure`, `SameSite=Lax` e `HttpOnly` quando esses atributos ainda não estão presentes.

O contador de requisições fica em um `Map` do processo. A chave é `req.ip`, com alternativas em `remoteAddress` e `unknown`. Cada entrada contém `count` e `resetAt`. A janela tem 60 segundos e permite dez requisições por chave. Quando a contagem já atingiu dez, o endpoint responde 429. Na gestão de capacidade, o mapa utiliza o limite de 5.000 entradas, remove entradas expiradas e aplica uma poda das mais antigas, preservando a chave da requisição corrente.

4.3 Rotas e entrega de arquivos

A rota de negócio é `POST /api/registrar-interesse`. Ela recebe o objeto do formulário e entrega a `persistLead` a decisão sobre o status HTTP e o corpo JSON da resposta. O cadastro é público e seu processamento ocorre inteiramente no servidor.

A rota `GET /.well-known/:file` serve manifestos de `public/.well-known`. O parâmetro é convertido em nome de arquivo por `path.basename`; o diretório é definido a partir do diretório de trabalho do processo. Um arquivo encontrado é enviado com tipo `application/json`. Para um manifesto inexistente, a resposta é 404, com `NOT_FOUND` e a mensagem “Manifesto não encontrado.”.

Em produção, `express.static` atende os artefatos de `dist`, com tratamento de arquivos iniciados por ponto configurado como `ignore`. O encaminhamento final devolve 404 JSON para recursos inexistentes sob `/api/`, `/.well-known/` e `/assets/`. As demais rotas recebem o HTML principal, permitindo que o roteador do frontend componha a página solicitada.

4.4 Contrato de entrada do cadastro

O contrato está em `src/types/leads.ts`, no `LeadRegistrationInputSchema`. O schema Zod define um objeto estrito, composto somente pelos campos declarados. Todos são strings; `cargo`, `curso` e `observacoes` têm valor padrão vazio quando omitidos. A transformação `trim` remove espaços nas extremidades antes da aplicação dos limites.

Campo	Presença	Transformação e regra
`nome`	Obrigatório	`trim`; mínimo 2, máximo 150 caracteres.
`email`	Obrigatório	`trim`, conversão para minúsculas, validação de endereço de email; máximo 150.
`telefone`	Obrigatório	`trim`; mínimo 8, máximo 30 caracteres.
`instituicao`	Obrigatório	`trim`; mínimo 2, máximo 200 caracteres.
`campus`	Obrigatório	`trim`; mínimo 1, máximo 100 caracteres.
`cargo`	Opcional	`trim`; máximo 100; padrão `''`.
`curso`	Opcional	`trim`; máximo 100; padrão `''`.
`previsaoSemestre`	Obrigatório	`trim`; mínimo 1, máximo 100 caracteres.
`observacoes`	Opcional	`trim`; máximo 1.000; padrão `''`.

O tipo `LeadRegistrationInput` é inferido desse schema. A função `persistLead` utiliza `safeParse`: o resultado válido contém os valores normalizados; o resultado inválido produz 400 e inclui os detalhes formatados pelo Zod. Assim, a estrutura enviada pelo formulário e a estrutura utilizada para montar os registros compartilham o mesmo contrato de entrada.

O email normalizado alimenta tanto o campo cifrado do cofre quanto o cálculo de unicidade. O telefone é tratado como texto dentro do intervalo declarado. A previsão de semestre também é uma string, utilizada no registro operacional conforme recebida após a normalização.

4.5 Respostas HTTP e integração com o cliente

Os resultados do cadastro são expressos por status HTTP e objetos JSON. As respostas de erro usam `error` e `message`; a validação do schema acrescenta `details`. O sucesso usa `success`, `message` e `id`.

Status	Identificador	Significado implementado
201	`success: true`	A gravação retornou sucesso; a resposta informa o identificador do lead.
400	`INVALID_ARGUMENT`	JSON malformado ou objeto rejeitado pelo schema.
409	`DUPLICATE`	O processamento encontrou o email no controle de unicidade.
413	`PAYLOAD_TOO_LARGE`	O parser recebeu um corpo acima do limite permitido.
429	`RATE_LIMIT_EXCEEDED`	A chave da requisição atingiu o limite na janela corrente.
500	`INTERNAL_ERROR`	Uma exceção foi capturada no processamento ou no handler geral.
503	`SERVICE_UNAVAILABLE`	A rotina de persistência retornou ausência de sucesso.

A mensagem de 201 é “Interesse registrado com sucesso!”. A resposta de 409 informa “E-mail ou registro já cadastrado.”. A de 503 descreve o serviço de persistência como temporariamente indisponível. A mensagem de 500 do processamento é “Erro interno ao processar requisição.”; o handler geral utiliza “Erro interno no servidor.”.

No frontend, `registrarManifestacaoInteresse`, em `src/api/leadsService.ts`, envia `fetch` para o endpoint com método POST, `Content-Type: application/json` e `JSON.stringify(data)`. A leitura da resposta tenta interpretar o JSON. Um 409 é convertido em `DuplicateLeadError`. Os demais resultados são aceitos quando a resposta HTTP é bem-sucedida e o objeto contém `success: true`; então o serviço devolve o indicador de sucesso e a mensagem para a interface.

4.6 Caminho completo de uma manifestação

Depois da leitura do JSON e da passagem pelo contador, o cadastro percorre esta sequência:

1. `persistLead` valida e normaliza o objeto com o schema Zod.
2. `executeLeadCreation` gera um identificador no formato `lead_<UUID>`.
3. `getEmailHash` calcula o HMAC do email normalizado.
4. `buildOperationalRecord` monta os dados da manifestação, seu estado e suas referências.
5. `buildVaultRecord` cifra nome, email, telefone e observações.
6. `writeLeadToBucket` procura uma parte e executa a operação Firestore correspondente.
7. O resultado de duplicidade, indisponibilidade ou sucesso é convertido em resposta HTTP.

O identificador gerado acompanha a manifestação, o documento do cofre e o índice. A resposta pública retorna esse identificador de lead; o nome da parte é utilizado pela organização interna dos dados. Os campos `origem` e `status` são definidos pelo construtor operacional como `portal_cra_sp` e `pendente_analise`.

5. Configuração Firebase, organização dos dados e criptografia

5.1 Seleção do projeto e da base Firestore

O servidor tenta ler `firebase-applet-config.json` no diretório de trabalho e interpretar seu JSON. A função de leitura devolve um objeto vazio quando a leitura ou a interpretação não é concluída. Os campos utilizados são `projectId` e `firestoreDatabaseId`.

Para o projeto, a precedência é `FIREBASE_PROJECT_ID`, depois `projectId` do arquivo e, por fim, `asg-ods`. Para a base, `FIRESTORE_DATABASE_ID` prevalece quando possui um valor diferente de `(default)`. Quando a variável está vazia ou contém `(default)`, o código utiliza `firestoreDatabaseId` do arquivo se esse campo estiver preenchido; depois considera o valor da variável.

Configuração de base	Resultado selecionado
Variável com nome de base	Nome informado pela variável.
Variável `(default)` e arquivo com nome de base	Nome informado pelo arquivo.
Variável `(default)` e arquivo sem base	Base padrão.
Variável ausente e arquivo com nome de base	Nome informado pelo arquivo.
Variável e arquivo sem base	Base padrão.

A inicialização reutiliza a primeira aplicação Firebase Admin existente ou cria uma aplicação com o projeto selecionado. Para uma base nomeada, chama `getFirestore(app, dbId)`; para a base padrão, chama `getFirestore(app)`. A instância resultante é a referência utilizada pela persistência. Uma exceção na inicialização é registrada e resulta em referência `null`.

A configuração Firebase Hosting publica `dist` e encaminha `/api/**` ao serviço Cloud Run `asg-ods`, na região `southamerica-east1`. O arquivo `.firebaserc` associa o projeto padrão ao mesmo identificador `asg-ods`. As regras de acesso de clientes do Firestore são declaradas em `firestore.rules`, com `allow read, write: if false` no caminho abrangente de documentos. O processamento do backend utiliza o SDK Admin.

5.2 Coleções e vínculos entre registros

A persistência distribui o cadastro entre três coleções. `leads_adesao` contém os documentos agregadores `parte1`, `parte2` e assim por diante. `leads_unicidade` contém um documento por HMAC de email. `vault_leads` contém um documento por identificador de lead.

A manifestação operacional, armazenada dentro de uma parte, contém estes campos:

Campo	Conteúdo
`leadId`	Identificador gerado para o cadastro.
`emailHash`	HMAC-SHA256 do email normalizado, em hexadecimal.
`instituicao`, `campus`, `cargo`, `curso`	Valores normalizados do formulário.
`previsaoSemestre`	Previsão informada no formulário.
`temObservacoes`	Booleano calculado a partir da presença de observações após `trim`.
`origem`	Constante `portal_cra_sp`.
`status`	Constante `pendente_analise`.

Campo	Conteúdo
`criadoEm`	Data e hora de criação em string ISO.
`vaultRef`	Caminho `vault_leads/<leadId>`.

O cofre possui `leadId`, os campos cifrados `nome`, `email`, `telefone` e `observacoes`, além de `criadoEm` e `vaultEncrypted: true`. O índice contém `criadoEm`, `parte` e `leadId`. Seu identificador é o próprio HMAC do email, e `parte` guarda o nome do documento agregador.

Exemplo exclusivamente esquemático dos vínculos, com marcadores simbólicos:

```
leads_unicidade/<HMAC_DO_EMAIL>
    parte -> parte<N>
    leadId -> <IDENTIFICADOR_DO_LEAD>

leads_adesao/parte<N>
    manifestacoes -> registros operacionais
    registro.vaultRef -> vault_leads/<IDENTIFICADOR_DO_LEAD>

vault_leads/<IDENTIFICADOR_DO_LEAD>
    campos de contato e observações -> envelopes cifrados
```

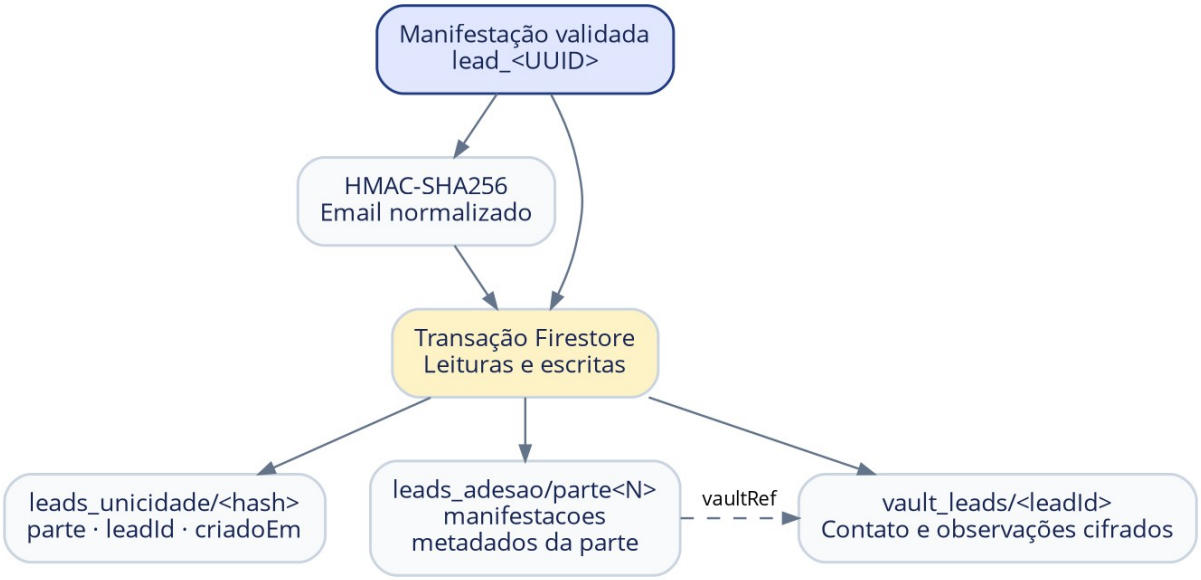


Figura 3 — Vínculos dos registros na transação Firestore.

5.3 Documentos agregadores e cálculo de capacidade

Cada parte possui `numeroParte`, `criadoEm`, `atualizadoEm`, `totalRegistros`, `tamanhoAproximadoBytes` e `manifestacoes`. O número acompanha o ordinal da parte; a lista `manifestacoes` reúne os registros operacionais. Ao inserir uma manifestação, `buildBucketPayload` constrói uma nova lista com os itens existentes e o novo registro. Preserva a criação original e atualiza a data da inclusão.

`totalRegistros` é o comprimento da lista resultante. `tamanhoAproximadoBytes` é calculado com `Buffer.byteLength(JSON.stringify(manifestacoes), 'utf8')`. O critério de encaixe utiliza a serialização do documento de parte existente, somada ao tamanho serializado do novo registro. O resultado é comparado com `MAX_BUCKET_BYTES`, definido como 943.718 bytes, correspondente ao teto de 90% de 1 MiB adotado pelo projeto.

A busca começa em `parte1`. Se uma parte existente comporta a manifestação, a rotina grava nela. Quando o critério de encaixe não é atendido, incrementa `partNum` e tenta a próxima. Uma parte ainda inexistente recebe sua estrutura inicial com a primeira manifestação.

5.4 Transação, unicidade e reconstrução do índice

Para cada tentativa, a rotina prepara referências ao documento de unicidade, à parte corrente e ao cofre do lead. A operação utiliza `db.runTransaction`. Primeiro lê o índice; quando ele existe, retorna duplicidade. Em seguida, lê a parte para decidir entre criar seu documento, acrescentar uma manifestação ou continuar a busca.

A conferência de duplicidade dentro de uma parte compara `emailHash`. Quando encontra uma manifestação correspondente, localiza seu `leadId` e grava novamente o índice com esse identificador e o nome da parte. A função utiliza o identificador preparado para a operação como alternativa quando o campo localizado está ausente. Esse caminho retorna duplicidade e o identificador selecionado.

Na criação de uma parte ou na inclusão em uma existente, `writeDualRecords` agenda `tx.set` para o índice, para o cofre recebido e para o documento agregador. O documento de parte é montado com a lista completa resultante. O mesmo contexto transacional reúne essas escritas e as leituras que orientaram a decisão.

`writeLeadToBucket` recebe o resultado da tentativa e encerra quando encontra sucesso ou duplicidade. Caso a parte não comporte o item, avança para a seguinte. Se a referência Firestore estiver `null`, devolve ausência de sucesso. Exceções da operação são tratadas nessa função, registradas com mensagem sanitizada e convertidas no mesmo resultado de persistência não concluída.

5.5 HMAC, cifragem e formato do cofre

O HMAC de unicidade utiliza SHA-256 e a variável `HMAC_PEPPER_KEY`. A função aplica minúsculas e `trim` ao email antes de calcular o valor hexadecimal. Esse valor é utilizado como identificador do documento de índice e como `emailHash` na manifestação operacional.

A cifragem utiliza `VAULT_ENCRYPTION_KEY`. O código aplica SHA-256 ao conteúdo dessa variável para obter a chave binária de 32 bytes usada pelo AES-256-GCM. Cada chamada gera um IV aleatório de 12 bytes, cifra a string em UTF-8 e obtém a tag de autenticação. O resultado é serializado neste envelope esquemático:

`v1:<IV_EM_BASE64>:<TAG_EM_BASE64>:<CONTEUDO_CIFRADO_EM_BASE64>`

Nome, email, telefone e observações recebem envelopes próprios. Uma string vazia é preservada como string vazia. `buildVaultRecord` reúne os resultados, o identificador, a data ISO e o indicador `vaultEncrypted`.

`decryptVaultField` interpreta o envelope `v1`, converte seus componentes de Base64 e executa a decifragem com a tag. Para entrada vazia, formato sem prefixo `v1:` ou quantidade diferente de quatro componentes, retorna o texto recebido. Uma exceção durante a decifragem resulta no marcador `[ENCRYPTED_VAULT_PAYLOAD]`. A ausência das variáveis exigidas para calcular HMAC ou cifrar campos lança uma exceção no processamento correspondente.

5.6 Mensagens operacionais e sanitização

As mensagens operacionais distinguem a inicialização Firestore, a gravação, o processamento do cofre e o handler geral. As últimas três utilizam os prefixos `[FIREBASE_WARN]`, `[SECURITY_VAULT_ERROR]` e `[SERVER_ERROR]`, respectivamente.

`sanitizeErrorMessage` começa selecionando os primeiros 2.000 caracteres do texto. Em sequência, substitui padrões de email, telefone, CPF e identificadores presentes em caminhos `leads_adesao/`. Os marcadores são `[REDACTED_EMAIL]`, `[REDACTED_PHONE]`, `[REDACTED_CPF]` e `leads_adesao/[REDACTED_DOC]`. A mensagem resultante segue para o console do processo. As respostas HTTP utilizam as mensagens do contrato do endpoint, mantendo a comunicação com o formulário organizada pelos códigos de resultado descritos no capítulo anterior.

6. Protocolos HTTP e recursos de distribuição

6.1 Cabeçalhos definidos pelo backend

O middleware `applySecurityHeaders`, em `src/server/securityHeaders.ts`, prepara os cabeçalhos de cada resposta Express antes da leitura do corpo e da seleção da rota. A escolha de CSP produtiva ou de desenvolvimento usa o valor de `NODE_ENV`. O conjunto inclui regras de transporte, tipo de conteúdo, informação de referência e interação entre origens.

Cabeçalho	Valor ou organização implementada
Strict-Transport-Security	`max-age=63072000; includeSubDomains; preload`.
X-Content-Type-Options	`nosniff`.
Referrer-Policy	`no-referrer`.
Cross-Origin-Opener-Policy	`same-origin-allow-popups`.
Cross-Origin-Embedder-Policy	`credentialless`.
Cross-Origin-Resource-Policy	`same-origin`.
Permissions-Policy	Câmera, microfone, geolocalização, pagamento, USB e browsing topics usam listas vazias.
Content-Security-Policy	String de diretivas obtida por `getActiveCsp()`.

O parâmetro `max-age` do HSTS é expresso em segundos. As diretivas `includeSubDomains` e `preload` fazem parte do valor configurado. `no-referrer` determina a política de informação de referência nas respostas, enquanto `nosniff` controla a interpretação do tipo de conteúdo. As políticas de origem e de capacidades são entregues como cabeçalhos HTTP pelo middleware.

O Express desabilita a exposição do cabeçalho X-Powered-By. A função de middleware mantém uma referência ao `setHeader` original e examina definições de Set-Cookie. Cada cookie recebe Secure, SameSite=Lax e HttpOnly somente quando o respectivo atributo ainda não existe no texto. Listas de cookies são processadas individualmente e preservadas como listas.

6.2 Organização da Content Security Policy

A CSP começa com `default-src 'none'` e declara fontes por tipo de recurso. A política produtiva contempla a origem da aplicação, os hashes declarados de conteúdo inline e os destinos externos utilizados pelas integrações. Os domínios de VLibras aparecem nas diretivas correspondentes a script, estilo, fonte, imagem, conexão, frame e worker.

Diretiva	Organização das fontes
script-src	Origem própria, hash SHA-256 declarado e destinos VLibras e GitHub CDN declarados no arquivo.
style-src	Origem própria, hashes declarados, Google Fonts e destinos da integração VLibras.
style-src-attr	Permite atributos inline de estilo.
font-src	Origem própria, fonts.gstatic.com e destinos declarados da integração.
img-src	Origem própria, blob/data e os destinos de imagem presentes na configuração.
connect-src	Origem própria, domínios Google APIs/Firebase e destinos VLibras declarados.

Diretiva	Organização das fontes
frame-src e worker-src	Conjuntos declarados para a integração e a própria origem; worker inclui blob.
object-src	none.
base-uri e form-action	self.
frame-ancestors	Origem própria e hosts declarados dos ambientes Google, run.app e AI Studio.
upgrade-insecure-requests	Diretiva declarada na política.

No desenvolvimento, a variante inclui as permissões utilizadas pela integração Vite e pelo hot module replacement. A configuração Firebase Hosting mantém seus próprios cabeçalhos no arquivo `firebase.json`, aplicados às respostas estáticas. Assim, a declaração do backend e a declaração do serviço de hospedagem estão localizadas em arquivos distintos do mesmo projeto.

6.3 Integridade dos assets e distribuição estática

A etapa `scripts/inject-sri.ts` completa o HTML compilado com integridade de sub-recursos, SRI. O cálculo utiliza os bytes dos arquivos referenciados pelas tags script e link do documento de entrada. SHA-384 é convertido para Base64 e identificado pelo prefixo `sha384-` no atributo integrity. O navegador recebe o HTML com o caminho do asset e seu valor de integridade.

A aplicação utiliza imagens WebP em banners e cartões e marcas institucionais em PNG/SVG. Os cinco logos das signatárias são recursos de `/imgs/partners`. Os caminhos de `public` tornam-se caminhos públicos no build. Os arquivos gerados em `dist/assets` recebem os nomes definidos pela compilação Vite; index.html referencia esses resultados.

O documento de entrada contém idioma pt-BR, viewport responsivo, descrição, identificação Open Graph, URL canônica, favicon e links para ARD e AI Catalog. Duas imagens têm preload condicionado por media query: uma para a composição desktop e outra para a composição mobile. Há também preconnect e dns-prefetch para o host VLibras.

6.4 Manifestos e endereços públicos

Os recursos textuais de `public` descrevem a navegação pública para ferramentas e consumidores externos. O sitemap relaciona as cinco páginas principais. robots.txt declara os caminhos do sitemap e a seleção de caminhos. llms.txt oferece identificação e links em formato textual; os catálogos JSON e o manifesto ARD apresentam os campos e capacidades declarados pelo projeto.

A rota Express `/.well-known/:file` atende os manifestos disponíveis na pasta correspondente. No Firebase Hosting, esses caminhos possuem rewrites próprios. A regra final de navegação devolve o documento da SPA, permitindo que React Router decida o conteúdo de uma rota pública aberta diretamente.

As referências dos resultados externos identificam o domínio avaliado. A relação entre cada ferramenta e seu resultado está no quadro inicial; os nomes dos cabeçalhos observados no relatório externo são apresentados como dados daquele relatório, enquanto este capítulo descreve os valores declarados pelo código de referência.

7. Operação, configuração e testes

Este capítulo descreve os comandos e arquivos operacionais do portal Componente Curricular de ASG e ODS, conforme o código da revisão `9a126c65ba902e19b31bc695be0ccc172df7167f`. A aplicação reúne frontend React, servidor Express, persistência com Firebase Admin e utilitários TypeScript. Os comandos são executados na raiz do checkout, onde ficam `package.json`, `package-lock.json`, `server.ts` e as configurações de compilação. O mesmo servidor Express atende a API e seleciona a forma de entrega da interface conforme o ambiente de execução.

7.1 Runtime e instalação

O manifesto declara Node.js **22.18.0 ou superior**, utiliza módulos ECMAScript por meio de `"type": "module"` e identifica o pacote como privado. A execução de produção chama diretamente `node server.ts`; a execução de desenvolvimento utiliza `tsx`. O TypeScript opera com alvo ES2022, módulos ESNext e resolução `bundler`. As opções `strict`, `noUnusedLocals` e `noUnusedParameters` estão habilitadas, e a verificação usa `noEmit`.

O npm é o gerenciador associado aos scripts e ao lockfile de versão 3. `package.json` declara intervalos de versões; `package-lock.json` registra a árvore resolvida e os metadados dos pacotes. Entre as dependências de aplicação estão Express, Firebase Admin, React, React DOM, React Router, Zod, Motion e Lucide React. As ferramentas de desenvolvimento incluem Vite, TypeScript, tsx, Vitest, Supertest, Tailwind CSS e plugins de integração com React e Vite.

A instalação do checkout segue o comando:

```
npm ci
```

Esse comando utiliza o manifesto e o lockfile para instalar a árvore de dependências em `node_modules`. Os executáveis locais passam a ser resolvidos automaticamente pelos scripts npm. A pasta de dependências, os diretórios `dist`, `build` e `coverage` e os arquivos de log estão entre as entradas de `.gitignore`.

7.2 Comandos disponíveis

Comando	Implementação declarada	Operação
`npm run dev`	`tsx server.ts`	Inicia Express com integração Vite no desenvolvimento.
`npm run build`	`vite build && tsx scripts/inject-sri.ts`	Compila o frontend e acrescenta integridade aos assets referenciados no HTML.
`npm run preview`	`vite preview`	Disponibiliza a prévia do frontend compilado pelo Vite.
`npm start`	`node server.ts`	Executa o servidor usando o runtime Node.
`npm test`	`vitest run`	Executa os arquivos de testes em uma rodada.
`npm run typecheck`	`tsc --noEmit`	Verifica os tipos sem emitir JavaScript.
`npm run lint`	`tsc --noEmit && tsx scripts/check-metrics.ts`	Encadeia tipos e processamento das métricas.
`npm run check:metrics`	`tsx scripts/check-metrics.ts`	Calcula métricas e grava o relatório Markdown.
`npm run codigofonte`	`tsx scripts/generate-codigofonte.ts`	Gera o agregado textual `CODIGOFONTE.txt`.
`npm run clean`	`rm -rf dist`	Remove o diretório de saída do build.

7.3 Desenvolvimento, build e produção

No desenvolvimento, `npm run dev` executa `server.ts`. Ao iniciar o servidor em ambiente diferente de `production` e de `test`, é criada uma instância Vite em `middlewareMode`, com `appType: 'spa'`, e incorpora seus middlewares ao Express. A aplicação continua usando suas rotas de API no mesmo processo.

`vite.config.ts` registra os plugins React e Tailwind CSS. O alias `@` aponta para a raiz do projeto, correspondendo ao mapeamento `@/*` de `tsconfig.json`. A configuração de desenvolvimento usa porta 3000, interface `0.0.0.0` e hosts permitidos com sufixos `.ai.studio`, `.run.app`, além de `localhost` e `127.0.0.1`. O servidor Express fornece essa mesma lista ao criar a instância Vite. A opção HMR é calculada pela expressão `process.env.DISABLE_HMR !== 'true'`.

O build gera `dist/index.html` e os assets compilados. Na segunda etapa, `inject-sri.ts` procura tags de script e link com caminhos em `/assets/`, lê os arquivos correspondentes e calcula SHA-384 em Base64. O atributo `integrity` é inserido ou atualizado no HTML. A etapa sinaliza erro quando o HTML ou um asset referenciado não é encontrado. A execução automática do utilitário é condicionada a `NODE_ENV` diferente de `test`.

O fluxo de produção é:

```
npm run build
NODE_ENV=production npm start
```

Express disponibiliza os arquivos de `dist` e entrega `dist/index.html` como resposta das rotas de navegação da SPA. Recursos não encontrados sob `/api/`, `/.well-known/` ou `/assets/` recebem resposta JSON 404. O serviço escuta em `0.0.0.0`; a porta é obtida por `Number(process.env.PORT) || 3000`. Assim, a infraestrutura pode fornecer `PORT`, enquanto 3000 é o valor padrão. Com `NODE_ENV=test`, o módulo exporta a aplicação para os testes sem chamar a rotina de inicialização do listener.

7.4 Firebase Hosting e Cloud Run

`firebase.json` define `dist` como diretório público do Firebase Hosting. As regras de exclusão listam a própria configuração e caminhos de Git, AI Studio e dependências. A seção de cabeçalhos aplica CSP, HSTS, política de referência, controles de origem e permissões às respostas abrangidas por `**`.

O rewrite de `/api/**` encaminha solicitações ao serviço Cloud Run **`asg-ods`**, na região **`southamerica-east1`**. Os manifestos `/.well-known/ard.json` e `/.well-known/ai-catalog.json`, assim como robots, sitemap, `llms.txt` e o catálogo da raiz, possuem entradas próprias de rewrite. A regra final direciona as demais rotas para `/index.html`, permitindo a navegação React a partir de URLs completas.

`.firebaserc` associa o alias `default` ao projeto `asg-ods`. A seção Firestore de `firebase.json` aponta para `firestore.rules`. `firebase-blueprint.json` descreve os documentos agregadores `leads_adesao`, incluindo número da parte, datas, total de registros, tamanho aproximado e lista de manifestações.

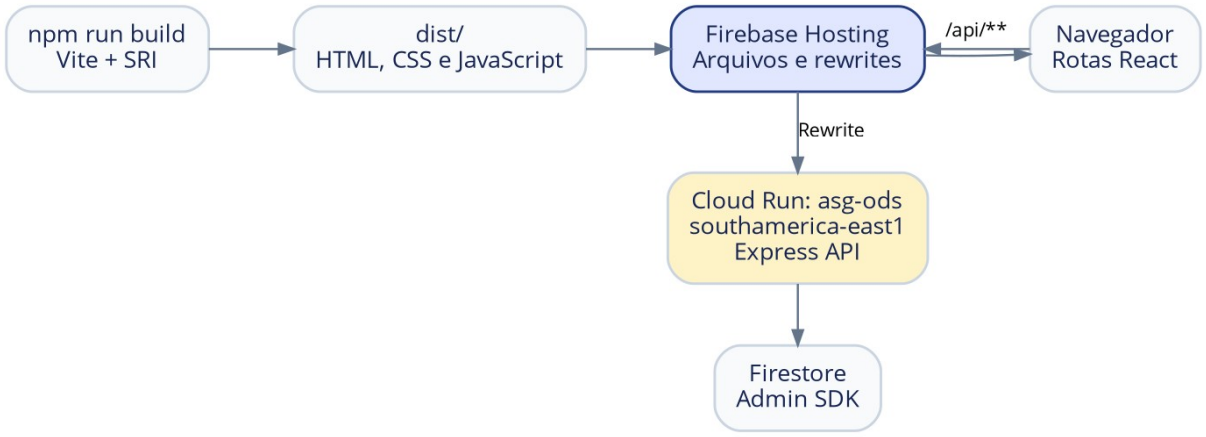


Figura 4 — Build, distribuição estática e encaminhamento da API.

7.5 Variáveis e credenciais

O backend lê suas configurações diretamente de `process.env` e, para identificar o projeto e banco, também consulta `firebase-applet-config.json`. Esse arquivo local utiliza os campos `projectId` e `firestoreDatabaseId`; quando sua leitura ou interpretação não produz configuração, o carregador retorna um objeto vazio.

Variável	Leitura e função na aplicação
`NODE_ENV`	Seleciona entrega de produção, inicialização em testes e variante dos cabeçalhos CSP. Também controla a execução automática do utilitário SRI.
`PORT`	Define a porta do listener Express, com padrão 3000.
`DISABLE_HMR`	Participa da configuração HMR do Vite; a string `true` resulta na opção desabilitada.
`FIREBASE_PROJECT_ID`	Tem precedência sobre `projectId` do arquivo local; o último padrão é `asg-ods`.
`FIRESTORE_DATABASE_ID`	Seleciona banco nomeado quando definido com valor diferente de `(default)`. Nos demais casos, participa da seleção junto ao arquivo local.
`HMAC_PEPPER_KEY`	Fornece a chave do HMAC-SHA256 usado para o índice derivado do e-mail normalizado. A função solicita sua presença ao calcular o índice.
`VAULT_ENCRYPTION_KEY`	Fornece o material do qual SHA-256 deriva a chave de 32 bytes usada pelo AES-256-GCM. A função solicita sua presença ao cifrar um campo preenchido.

Na seleção do banco, um identificador nomeado vindo da variável tem prioridade. Caso contrário, o código considera `firestoreDatabaseId` do arquivo e o valor da variável. O Admin SDK recebe `getFirestore(app, dbId)` para bancos nomeados e `getFirestore(app)` para o banco padrão.

O SDK é inicializado com o identificador do projeto e utiliza o mecanismo de Application Default Credentials, ou ADC, para a identidade do backend. Em um runtime gerenciado, essa identidade pode ser fornecida pela conta de serviço associada à execução. Em outro ambiente, a infraestrutura pode disponibilizar credenciais ADC por seu mecanismo seguro, inclusive por um arquivo associado a `GOOGLE_APPLICATION_CREDENTIALS`. As credenciais e as duas chaves de

aplicação são provisionadas no ambiente do processo servidor por configuração e gestão de segredos.

`.env.example` reúne os nomes de configuração e descrições. Além das variáveis Firebase e criptográficas usadas, contém `GEMINI_API_KEY`, `APP_URL`, `FIREBASE_API_KEY`, `FIREBASE_AUTH_DOMAIN`, `FIREBASE_STORAGE_BUCKET`, `FIREBASE_MESSAGING_SENDER_ID` e `FIREBASE_APP_ID`. Esses nomes não possuem leitura direta nos módulos atuais da aplicação. `GOOGLE_CLOUD_PROJECT` também não participa da expressão de seleção do projeto em `server.ts`. `.gitignore` exclui `.env*`, preservando a inclusão de `.env.example`, e exclui `firebase-applet-config.json`.

7.6 Organização dos testes

Os testes utilizam Vitest e estão distribuídos em **quatro arquivos, com 31 casos**. Supertest envia requisições à aplicação Express exportada, e os testes do cliente HTTP configuram respostas de `fetch` durante seus cenários. As funções de normalização, agregação, criptografia e métricas são chamadas diretamente.

Arquivo	Casos	Comportamentos descritos pelos testes
`tests/leads.test.ts`	20	Schema, cliente HTTP, respostas Express, limitação de tráfego, manifesto ARD, sanitização, buckets, cabeçalhos e SRI.
`tests/check-metrics.test.ts`	5	Classificação declarativa, complexidade, linhas lógicas e métricas do repositório.
`tests/vault.test.ts`	4	Presença das chaves, cifra e decifra, índice HMAC e separação entre cofre e registro operacional.
`tests/firestore-rules.test.ts`	2	Inspeção textual das regras Firestore e declaração de negação de leitura e escrita.

Em `leads.test.ts`, os casos Zod verificam normalização do e-mail, campos não declarados e entradas incompletas. O cliente trata sucesso 201, duplicidade 409, erro 500 e resposta sem o marcador `success: true`. O backend é exercitado com entrada inválida, JSON malformado, persistência indisponível e limite de tráfego atingido.

Os casos de limitação manipulam o mapa de contadores para verificar expiração, limite de entradas e preservação do IP corrente durante a poda. A sanitização é examinada com e-mail, telefone e referência documental. Os testes de bucket verificam o teto de 943.718 bytes, a criação da primeira parte e a inclusão de outra manifestação preservando a data de criação. O manifesto ARD é solicitado por HTTP e examinado como JSON.

Os cabeçalhos são conferidos em uma requisição sob a configuração de produção, incluindo CSP, HSTS e ausência de `x-powered-by`. Quando `dist/index.html` existe, o caso SRI procura o atributo `integrity` com prefixo SHA-384. Em `vault.test.ts`, os testes comparam duas cifras do mesmo texto, conferem a restauração do conteúdo, a estabilidade do HMAC após normalização e os campos presentes em cada modelo de registro.

7.7 Métricas e geração de relatórios

`check-metrics.ts` chama `runMetricsAudit()`, grava `RELATORIO-CONFORMIDADE-METRICAS.md` e apresenta o resultado no terminal. O processamento percorre a árvore em busca de `.ts`, `.tsx` e `.js`, ignorando diretórios chamados `node_modules`, `dist`, `build`, `.git`, `coverage` e `functions`.

O analisador usa a AST do TypeScript para identificar funções, métodos, construtores e acessores. Registra tamanho físico, linhas lógicas, complexidade ciclomática e cognitiva, além do

aninhamento. Os limites implementados são 250 linhas por arquivo de lógica, 120 caracteres por linha, 40 linhas físicas e 25 lógicas por função, CC até 7, cognitiva até 15 e aninhamento até três níveis. A classificação declarativa procura os nós imperativos definidos no utilitário, como condições, laços, `switch`, `try` e `throw`; a exceção de tamanho do arquivo usa o resultado dessa classificação.

As linhas lógicas são obtidas do corpo da função, excluindo linhas vazias, comentários e os delimitadores reconhecidos pelo contador. As funções internas possuem medição própria. O resultado reúne máximos, caminhos, violações e um livro de registros. O relatório Markdown apresenta data, matriz e até 30 funções ordenadas por complexidade e linhas lógicas. Se a lista de erros não estiver vazia, o comando termina com código 1 depois de gravar o relatório.

7.8 Agregado de código e arquivos públicos

`generate-codigofonte.ts` seleciona uma lista explícita de arquivos da raiz e percorre `src`, `scripts`, `tests` e `public`. A lista é ordenada antes da concatenação. Na travessia, ignora links simbólicos e entradas iniciadas por ponto. A seleção por nome aplica os padrões credential, secret, `.env`, token e extensões PEM e KEY. Uma lista de extensões separa formatos de imagens, arquivos compactados, PDF e fontes.

Após a leitura textual, o filtro de conteúdo procura os padrões configurados para chaves privadas, formatos Google e algumas atribuições de senha ou chave. Um arquivo identificado por esse filtro é omitido, com aviso no terminal. Cada bloco exportado recebe o cabeçalho `ARQUIVO: caminho`, o conteúdo e um separador de 60 sinais de igualdade. O resultado é gravado como `CODIGOFONTE.txt` na raiz.

Os arquivos de `public` acompanham o build com seus caminhos públicos. Há imagens WebP, marcas de parceiros em PNG/SVG e `favicon.svg`. Os manifestos ARD e AI Catalog usam `specVersion`, identificador, nome, capacidades e consultas representativas. `robots.txt` permite a navegação geral, declara `/api/` como caminho excluído e lista dois endereços de sitemap. `sitemap.xml` registra as cinco páginas públicas; `llms.txt` oferece os mesmos destinos em Markdown.

7.9 Catálogo operacional de arquivos

Grupo	Arquivos e finalidade
Inicialização	`server.ts`: servidor; `index.html`: documento de entrada; `metadata.json`: identificação do aplicativo.
Dependências e ferramentas	`package.json`, `package-lock.json`, `tsconfig.json`, `vite.config.ts`: scripts, resolução de pacotes, tipos e build.
Configuração de ambiente	`.env.example`, `.gitignore`, `.firebaserc`: nomes de variáveis, exclusões e alias Firebase.
Firebase	`firebase.json`, `firestore.rules`, `firebase-blueprint.json`: publicação, regras e descrição dos agregadores.
Governança e artefatos	`AGENTS.MD`, `RELATORIO-CONFORMIDADE-METRICAS.md`, `CODIGOFONTE.txt`: contrato do projeto e saídas dos utilitários.
Scripts de métricas	`check-metrics.ts`, `metrics-runner.ts`, `metrics-ast.ts`, `metrics-report.ts`, `metrics-types.ts`: comando, travessia, medição, apresentação e estruturas de resultado.

Grupo	Arquivos e finalidade
Outros scripts	`inject-sri.ts`, `generate-codigofonte.ts`: enriquecimento do HTML e agregado textual.
Testes	Os quatro arquivos em `tests`: cenários HTTP, regras, criptografia e métricas.
Contratos	`src/types/leads.ts`: schema Zod de entrada, tipo inferido e interface de registro; `scripts/metrics-types.ts`: interfaces das medições.

O diretório `.aistudio` mantém um plano de publicação e seus metadados. `dist` contém a saída de frontend preparada pelo build; `public` concentra os recursos estáticos de origem. Esses grupos complementam os módulos de frontend e backend descritos nos demais capítulos.

8. Glossário e referências de implementação

8.1 Termos utilizados

Termo	Uso neste documento
ASG	Ambiental, Social e Governança.
ODS	Objetivos de Desenvolvimento Sustentável.
IES	Instituição de Educação Superior.
SPA	Aplicação de página única; React compõe as telas a partir do documento de entrada.
API	Interface HTTP utilizada pela manifestação de interesse.
Schema	Estrutura declarada no Zod para validar o objeto de entrada.
Payload	Corpo JSON enviado ou recebido em uma operação.
Middleware	Etapa do pipeline de processamento Express.
Bucket / parte	Documento agregador de manifestações em leads_adesao.
Cofre	Documento de vault_leads que reúne campos cifrados.
HMAC	Código calculado com hash e chave; neste projeto, compõe o índice do email normalizado.
AES-GCM	Algoritmo e modo de cifragem dos campos do cofre.
IV	Valor aleatório utilizado por uma chamada de cifragem.
Tag	Valor de autenticação produzido pelo GCM.
ADC	Application Default Credentials, mecanismo de identidade do SDK Admin.
CSP	Content Security Policy, conjunto de fontes e diretivas de recursos.
SRI	Subresource Integrity, atributo de integridade dos assets compilados.
HMR	Hot Module Replacement, atualização de módulos no desenvolvimento.
ARD / AI Catalog	Recursos descritivos JSON publicados pelo projeto para navegação e descoberta.

8.2 Fontes de implementação

As descrições correspondem ao commit **9a126c65ba902e19b31bc695be0ccc172df7167f**. O repositório de referência é [desenvolvimentoprofissional-oss/asg-ods].

Área	Arquivos e grupos principais
Entrada e composição	index.html; src/main.tsx; src/App.tsx; src/routes/index.tsx.
Páginas e interface	src/pages; src/components/layout; home; chancela; metodologia; trilha; presentation; partners; ui; interest; accessibility.
Estado e preferências	src/store/useAccessibilityStore.ts; src/hooks/useAccessibilitySync.ts; src/index.css.
Conteúdo	src/constants: módulos, objetivos, cursos, parceiros, textos e navegação.
Contrato e cliente HTTP	src/types/leads.ts; src/api/leadsService.ts.
Backend e dados	server.ts; src/server/leadBucketService.ts; vaultService.ts; securityHeaders.ts.
Configuração e distribuição	package.json; package-lock.json; vite.config.ts; tsconfig.json; firebase.json; .firebaserc; firestore.rules; public.
Utilitários e testes	scripts; tests; AGENTS.MD; artefatos textuais gerados pelos comandos.

8.3 Referências das ferramentas externas

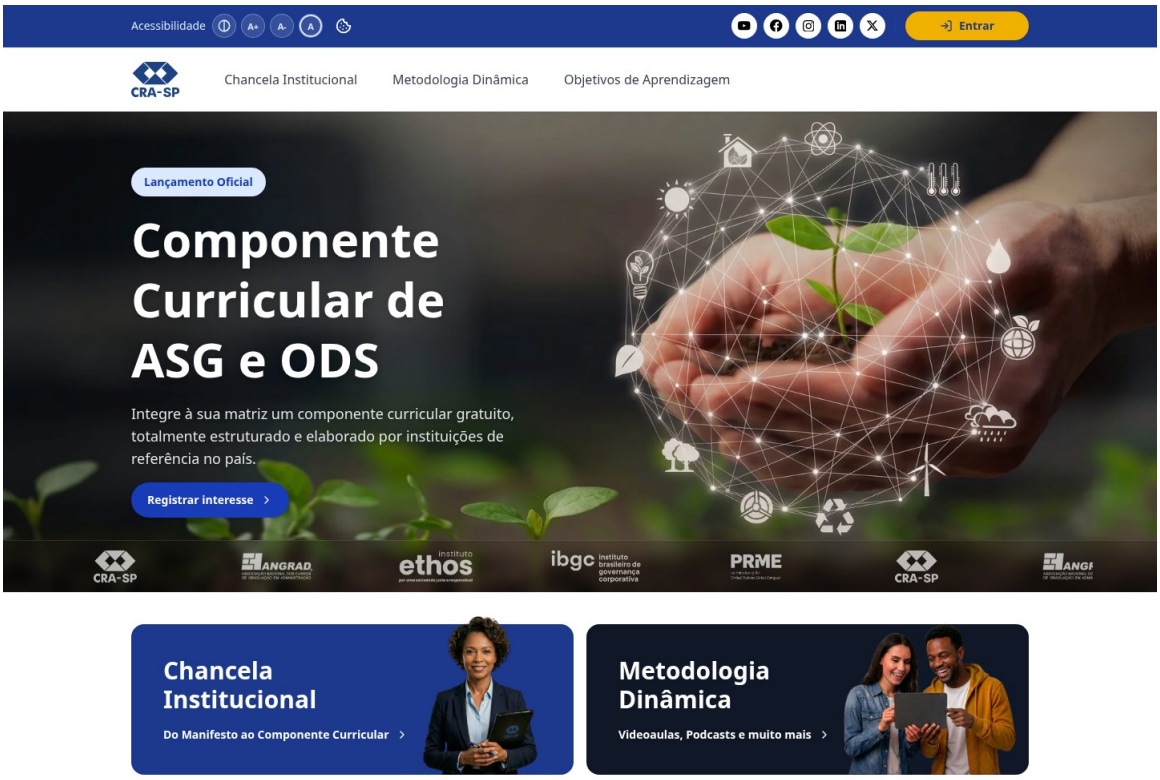
Os valores do capítulo inicial têm como fonte as capturas recebidas em 13 de outubro de 2026. Os endereços abaixo identificam as ferramentas e as páginas de avaliação correspondentes quando o endereço está disponível na captura.

1. MDN HTTP Observatory — asg-ods.ai.studio.
2. PageSpeed Insights — mobile.
3. PageSpeed Insights — desktop.
4. SecurityHeaders — asg-ods.ai.studio.
5. Catchpoint / WebPageTest — portal da ferramenta.
6. WAVE — asg-ods.ai.studio.
7. Webbkoll — asg-ods.ai.studio.

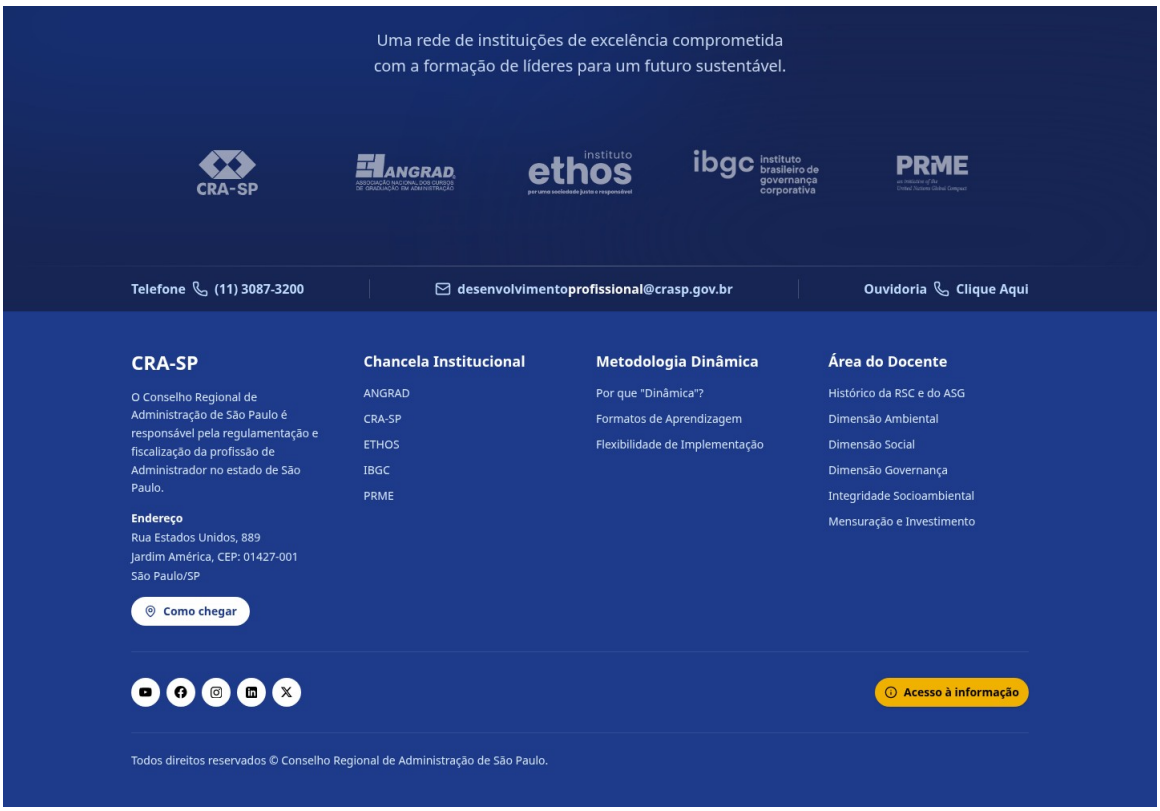
Os anexos de interface foram produzidos a partir da renderização local do build do mesmo commit, em 13/10/2026. As legendas identificam página, caminho e composição desktop ou mobile. Os dados do formulário permanecem vazios nessas figuras.

Anexo A. Páginas da aplicação — desktop

A.1 Página inicial



Página inicial · / · Desktop 1440 × 1000 · vista inicial.



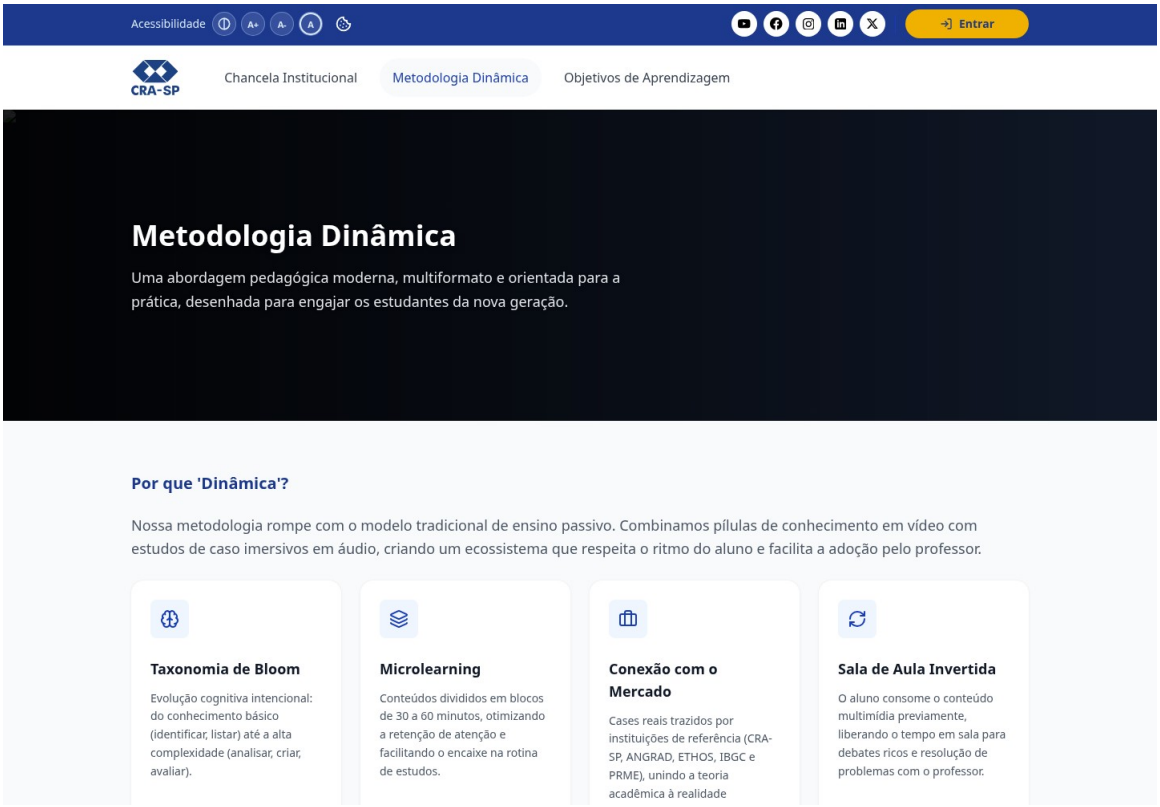
Página inicial · / · Desktop 1440 × 1000 · vista inferior.

A.2 Chancela institucional

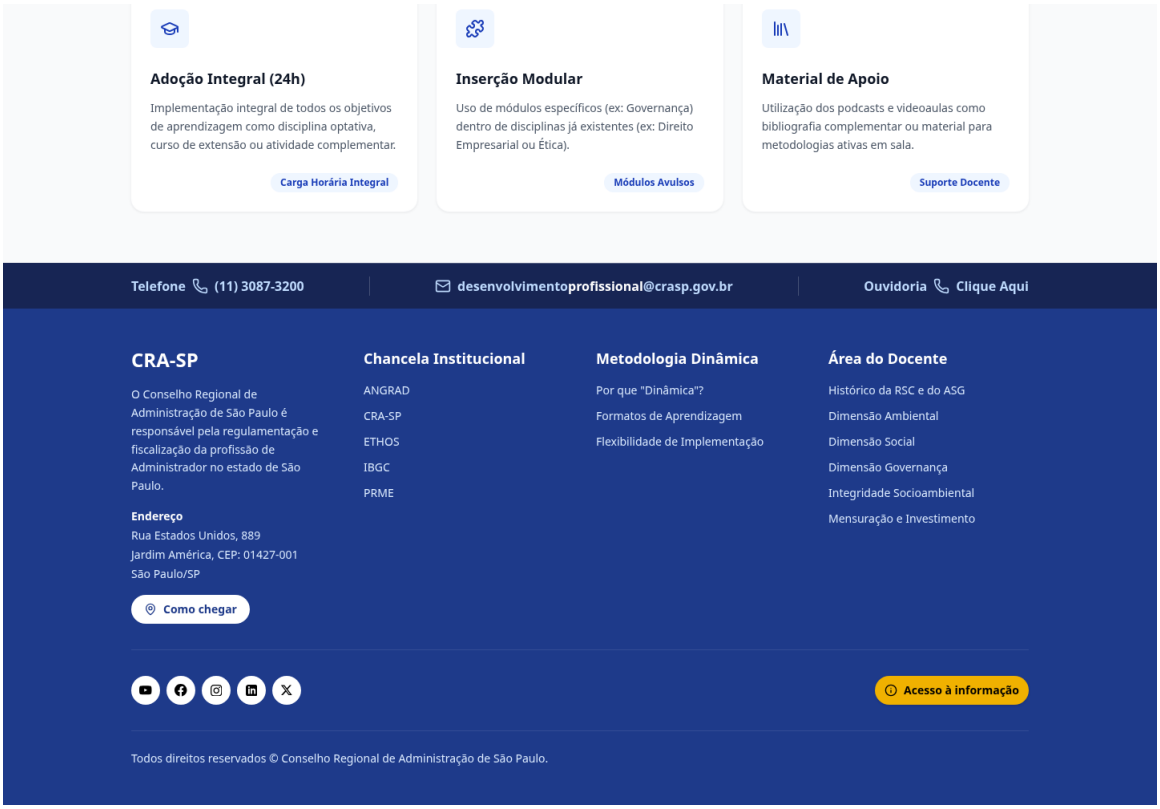


Chancela institucional · /chancela · Desktop 1440 × 1000 · vista inferior.

A.3 Metodologia dinâmica

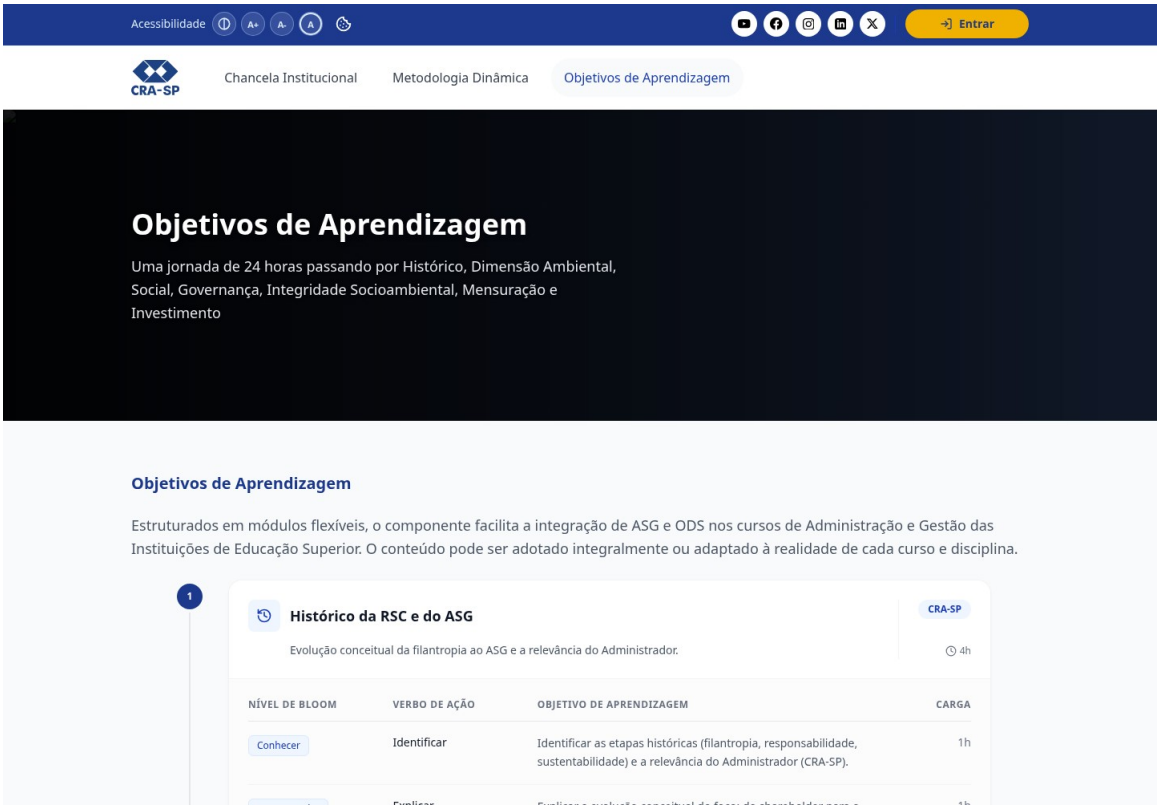


Metodologia dinâmica · /metodologia · Desktop 1440 × 1000 · vista inicial.

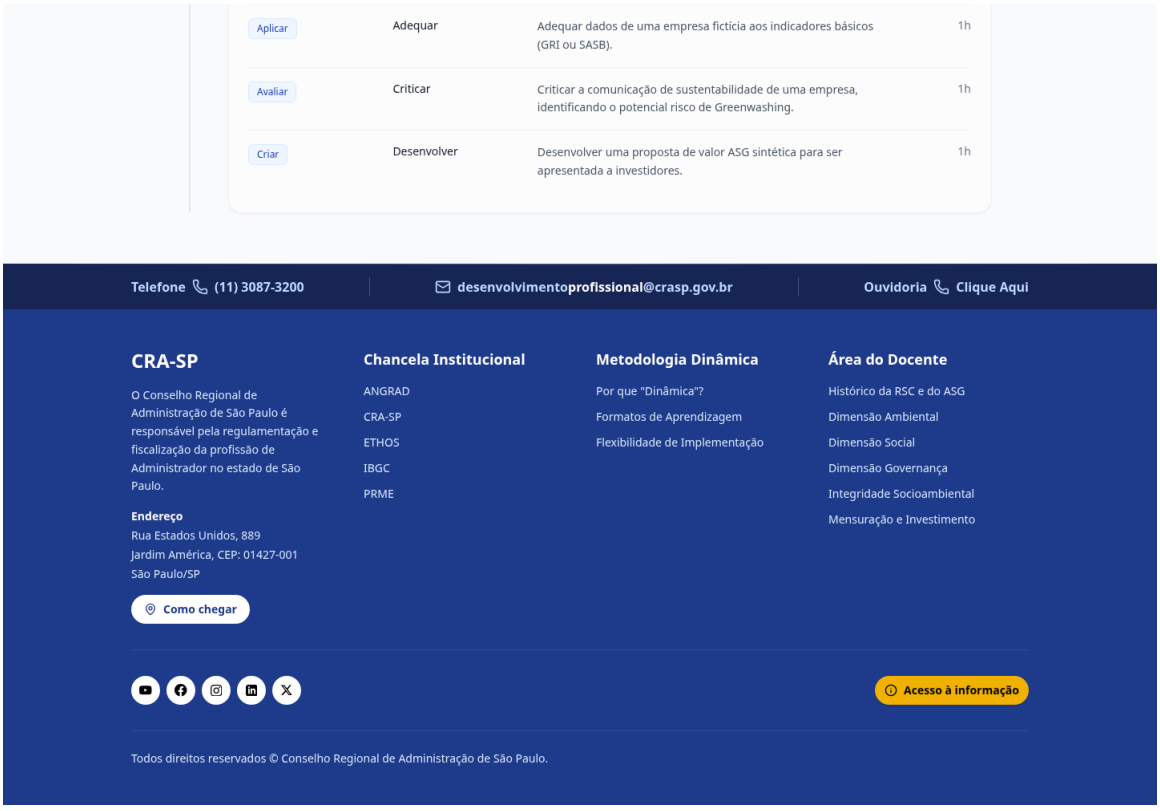


Metodologia dinâmica · /metodologia · Desktop 1440 × 1000 · vista inferior.

A.4 Objetivos de aprendizagem



Objetivos de aprendizagem · /trilha · Desktop 1440 × 1000 · vista inicial.



Objetivos de aprendizagem · /trilha · Desktop 1440 × 1000 · vista inferior.

A.5 Manifestação de interesse



Manifestação de interesse · /registro-interesse · Desktop 1440 × 1000 · vista inicial.



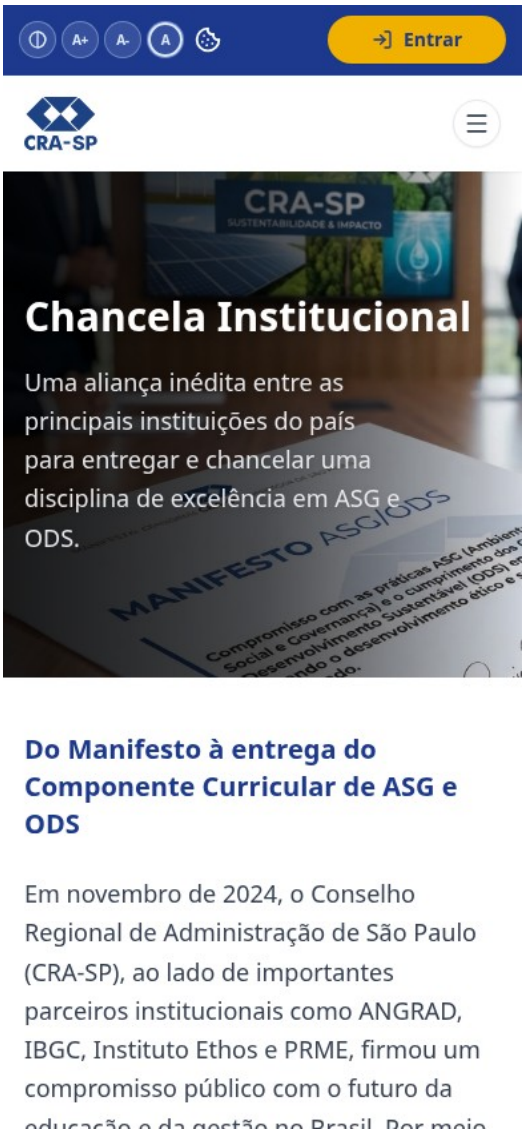
Manifestação de interesse · /registro-interesse · Desktop 1440 × 1000 · vista inferior.

Anexo B. Páginas da aplicação — mobile

B.1 Página inicial e Chancela institucional

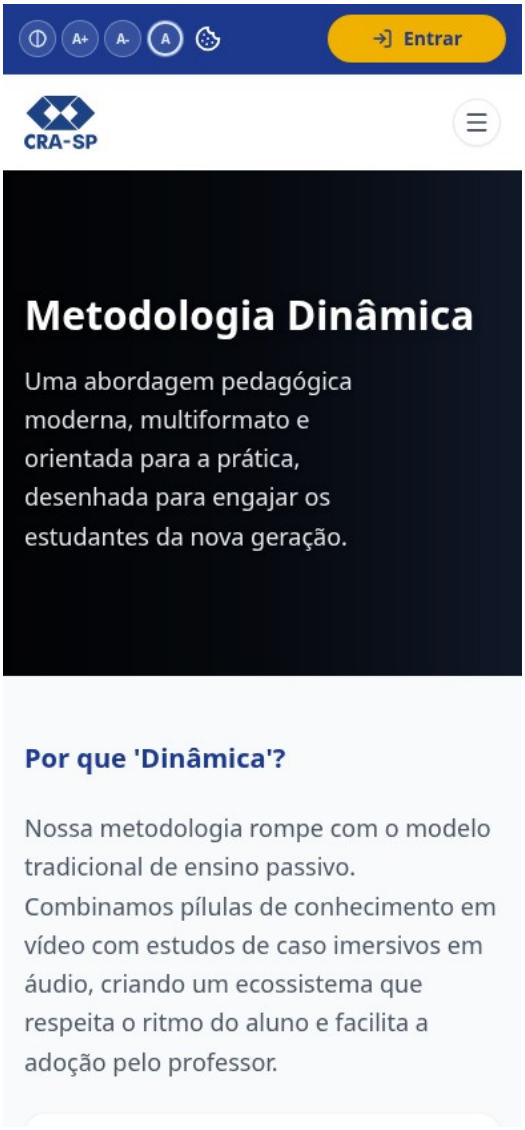


Página inicial
/
Mobile 390 × 844



Chancela institucional
/chancela
Mobile 390 × 844

B.2 Metodologia dinâmica e Objetivos de aprendizagem

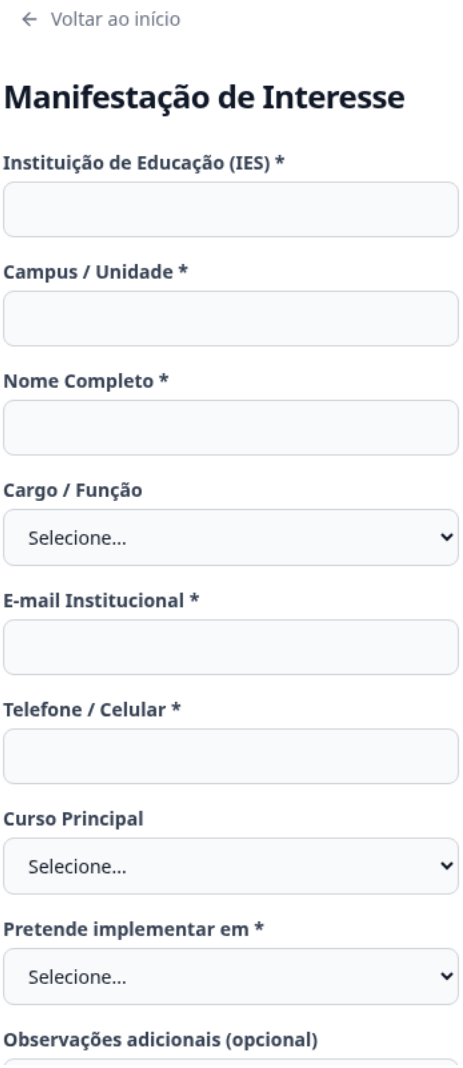


Metodologia dinâmica
/metodologia
Mobile 390 × 844



Objetivos de aprendizagem
/trilha
Mobile 390 × 844

B.3 Manifestação de interesse



Manifestação de interesse
/registro-interesse
Mobile 390 × 844